02장 파이썬 프로그래밍의 기초, 자료형

'어떤 프로그래밍 언어이든 그 언어의 자료형을 알고 이해할 수 있다면 이미 그 언어의 절반을 터득한 것이나 다름없다'라는 말이 있다. 자료형이란 프로그래밍을 할 때 쓰이는 숫자, 문자열 등과 같이 자료 형태로 사용하는 모든 것을 뜻한다. 프로그램의 기본이자 핵심 단위가 바로 자료형이다. 계산 프로그램을 만들려면 어떤 것을 계산할 것인지부터 알아야 하고 데이터베이스 프로그램을 만들려면 어떤 자료를 저장할 것인지부터 알아야 하는 것처럼 기본 중의 기본이다. 따라서 자료형을 충분히 이해하지 않고 프로그래밍을 시작하려는 것은 기초 공사가 마무리되지 않은 상태에서 빌딩을 세우는 것과 같다.

02장에서는 파이썬의 주요 자료형을 하나씩 알아본다.

목차

- 02-1 숫자형
- 02-2 문자열 자료형
- 02-3 리스트 자료형
- 02-4 튜플 자료형
- 02-5 딕셔너리 자료형
- 02-6 집합 자료형
- 02-7 불 자료형
- 02-8 자료형의 값을 저장하는 공간, 변수
- 02장 되새김 문제

02-1 숫자형

숫자형(Number)은 숫자 형태로 이루어진 자료형이다. 정수, 실수, 8진수, 16진수 등이 여기에 해당한다.

다음 표는 파이썬에서 숫자를 사용하는 예이다.

항목	파이썬 사용 예
정수	123, -345, 0
실수	123.45, -1234.5, 3.4e10
8진수	0o34, 0o25
16진수	0x2A, 0xFF

이제 숫자를 파이썬에서 어떻게 만들고 사용하는지 자세히 알아보자.

- 숫자형은 어떻게 만들고 사용할까?
  - 정수형
  - 실수형
  - 8진수와 16진수
- 숫자형을 활용하기 위한 연산자
  - 사칙 연산
  - x의 y제곱을 나타내는 ** 연산자
  - 나눗셈 후 나머지를 반환하는 % 연산자
  - 나눗셈 후 몫을 반환하는 // 연산자
  - 복합 연산자

숫자형은 어떻게 만들고 사용할까?

정수형

정수형(integer)은 말 그대로 정수를 뜻하는 자료형이다. 다음은 양의 정수, 음의 정수, 숫자 0을 변수 a에 대입하는 예이다.

```
>>> a = 123
>>> a = -178
>>> a = 0
```

실수형

실수형(floating-point)은 소수점이 포함된 숫자를 말한다. 다음은 실수를 변수 a에 대입하는 예이다.

```
>>> a = 1.2
>>> a = -3.45
```

> *정수 1과 실수 1.0은 값은 같지만 자료형이 다르다.*

아주 크거나 아주 작은 숫자를 표현할 때는 지수 표현 방식을 사용하면 편리하다. 파이썬에서는 4.24e10 또는 4.24E10처럼 표현할 수 있다(e와 E 둘 중 어느 것을 사용해도 된다).

```
>>> a = 4.24E10
>>> a = 4.24e-10
```

여기서 4.24E10은 $4.24 * 10^{10}$, 4.24e-10은 $4.24 * 10^{-10}$을 의미한다.

8진수와 16진수

8진수(octal)는 0o 또는 0O(숫자 0 + 알파벳 소문자 o 또는 대문자 O)로 시작한다.

```
>>> a = 0o177
>>> print(a)
127
```

> *0o177 = $1×8^2+7×8^1+7=127$*

16진수(hexadecimal)는 0x로 시작한다.

```
>>> a = 0x8ff
>>> b = 0xABC
>>> print(b)
2748
```

> *0xABC = $10×16^2+11×16^1+12=2748$ (A:10, B:11, C:12)*

8진수나 16진수는 파이썬에서 자주 사용하지 않으므로 간단히 눈으로만 익히고 넘어가자. 다만 파일 권한이나 색상 코드 등에서 가끔 볼 수 있으므로 이런 표현이 있다는 정도만 기억해 두자.

숫자형을 활용하기 위한 연산자

사칙 연산

프로그래밍을 처음 접하더라도 사칙 연산(`+` , `-` , `*` , `/`)은 알고 있을 것이다. 파이썬도 계산기처럼 연산자를 사용해 사칙 연산을 수행한다.

```
>>> a = 3
>>> b = 4
>>> a + b
7
>>> a - b
-1
>>> a * b
12
>>> a / b
0.75
```

x의 y제곱을 나타내는 `**` 연산자

`**` 연산자는 `x ** y` 처럼 사용하면 x의 y제곱(x^y) 값을 반환한다.

```
>>> a = 3
>>> b = 4
>>> a ** b
81
```

나눗셈 후 나머지를 반환하는 % 연산자

`%`는 나눗셈의 나머지 값을 반환하는 연산자이다. 7을 3으로 나누면 나머지는 1, 3을 7로 나누면 나머지는 3이 된다.

```
>>> 7 % 3
1
>>> 3 % 7
3
```

> *3을 7로 나누면 몫이 0이고 나머지가 3이므로 `3 % 7` 의 결과는 3이다.*

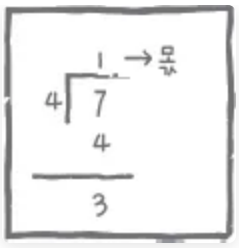


나눗셈 후 몫을 반환하는 // 연산자

`/` 연산자를 사용하여 7 나누기 4를 하면 결과는 1.75가 된다. 반면 `//` 연산자를 사용하면 나눗셈 후 몫만 반환한다. 두 연산자의 차이를 비교해 보자.

```
>>> 7 / 4
1.75
>>> 7 // 4
1
```

`/`는 소수점까지 포함한 결과를, `//`는 몫에 해당하는 정수값 1만 반환한다.



복합 연산자

복합 연산자는 산술 연산자(`+` , `-` 등)와 대입 연산자(`=`)를 합친 것이다.

먼저 a에 1을 대입한다.

```
>>> a = 1
```

a의 값을 1만큼 증가시키려면 다음과 같이 할 수 있다.

```
>>> a = a + 1  # a + 1로 계산된 값을 다시 a에 대입한다.
>>> print(a)
2
```

`a = a + 1` 은 다음과 같이 간략하게 표현할 수 있다.

```
>>> a += 1
```

`a += 1` 은 `a = a + 1` 과 동일하다. 이처럼 `+=` 를 복합 연산자라 부른다.

a의 값을 1만큼 감소시키려면 `-=` 복합 연산자를 사용한다.

```
>>> a = 1
>>> a -= 1  # a = a - 1과 같음
>>> print(a)
0
```

복합 연산자에는 다음과 같은 것들이 있다.

```
+=, -=, *=, /=, //=, %=, **=
```

예를 들어 `*=` 는 다음과 같이 사용한다.

```
>>> a = 2
>>> a *= 3  # a = a * 3과 같음
>>> print(a)
6
```

복합 연산자를 사용하면 코드를 더 간결하게 작성할 수 있다.

02-2 문자열 자료형

문자열(string)은 연속된 문자들의 나열이다. 예를 들면 다음과 같다.

```
"Life is too short, You need Python"
"a"
"123"
```

모든 예문이 큰따옴표(")로 둘러싸여 있다. '123은 숫자인데 왜 문자열이지?'라는 의문이 들 수 있다. 따옴표로 둘러싸여 있으면 모두 문자열이다.

- 문자열은 어떻게 만들고 사용할까?
  - 1. 큰따옴표로 양쪽 둘러싸기
  - 2. 작은따옴표로 양쪽 둘러싸기
  - 3. 큰따옴표 3개를 연속으로 써서 양쪽 둘러싸기
  - 4. 작은따옴표 3개를 연속으로 써서 양쪽 둘러싸기
- 문자열 안에 작은따옴표나 큰따옴표를 포함시키고 싶을 때
  - 1. 문자열에 작은따옴표 포함하기
  - 2. 문자열에 큰따옴표 포함하기
  - 3. 역슬래시를 사용해서 작은따옴표와 큰따옴표를 문자열에 포함하기
- 여러 줄인 문자열을 변수에 대입하고 싶을 때
  - 1. 줄을 바꾸기 위한 이스케이프 코드 \n 삽입하기
  - 2. 연속된 작은따옴표 3개 또는 큰따옴표 3개 사용하기
- 문자열 연산하기
  - 문자열 더해서 연결하기
  - 문자열 곱하기
  - 문자열 곱하기 응용하기
- 문자열 길이 구하기
- 문자열 인덱싱과 슬라이싱
  - 문자열 인덱싱
  - 문자열 인덱싱 활용하기
  - 문자열 슬라이싱
  - 문자열을 슬라이싱하는 방법
  - 슬라이싱으로 문자열 나누기
- 문자열 포매팅이란?
- 문자열 포매팅 따라 하기

    - 1. 숫자 바로 대입
    - 2. 문자열 바로 대입
    - 3. 숫자 값을 나타내는 변수로 대입
    - 4. 2개 이상의 값 넣기
- 문자열 포맷 코드
- 포맷 코드와 숫자 함께 사용하기
    - 1. 정렬과 공백
    - 2. 소수점 표현하기
- format 함수를 사용한 포매팅
    - 숫자 바로 대입하기
    - 문자열 바로 대입하기
    - 숫자 값을 가진 변수로 대입하기
    - 2개 이상의 값 넣기
    - 이름으로 넣기
    - 인덱스와 이름을 혼용해서 넣기
    - 왼쪽 정렬
    - 오른쪽 정렬
    - 가운데 정렬
    - 공백 채우기
    - 소수점 표현하기
    - { 또는 } 문자 표현하기
- f 문자열 포매팅
- 문자열 관련 함수들
    - 문자 개수 세기 - count
    - 위치 알려 주기 1 - find
    - 위치 알려 주기 2 - index
    - 문자열 삽입 - join
    - 소문자를 대문자로 바꾸기 - upper
    - 대문자를 소문자로 바꾸기 - lower
    - 왼쪽 공백 지우기 - lstrip
    - 오른쪽 공백 지우기 - rstrip
    - 양쪽 공백 지우기 - strip
    - 문자열 바꾸기 - replace
    - 문자열 나누기 - split
    - 문자열이 알파벳으로만 구성되어 있는지 확인하기 - isalpha
    - 문자열이 숫자로만 구성되어 있는지 확인하기 - isdigit
    - 문자열이 특정 문자(열)로 시작하는지 확인하기 - startswith
    - 문자열이 특정 문자(열)로 끝나는지 확인하기 - endswith

문자열은 어떻게 만들고 사용할까?

파이썬에서 문자열을 만드는 방법은 총 4가지다.

1. 큰따옴표로 양쪽 둘러싸기

```
"Hello World"
```

2. 작은따옴표로 양쪽 둘러싸기

```
'Python is fun'
```

3. 큰따옴표 3개를 연속으로 써서 양쪽 둘러싸기

```
"""Life is too short, You need python"""
```

4. 작은따옴표 3개를 연속으로 써서 양쪽 둘러싸기

```
'''Life is too short, You need python'''
```

단순함이 자랑인 파이썬이 문자열을 만드는 방법은 왜 4가지일까? 그 이유를 알아보자.

문자열 안에 작은따옴표나 큰따옴표를 포함시키고 싶을 때

문자열 안에 작은따옴표와 큰따옴표가 들어 있어야 할 경우가 있다. 이때는 특별한 방법이 필요하다. 예제를 하나씩 살펴보면서 원리를 익혀 보자.

1. 문자열에 작은따옴표 포함하기

```
Python's favorite food is perl
```

위 문자열을 food 변수에 저장하려면 문자열을 큰따옴표로 둘러싸야 한다. 큰따옴표 안에 들어 있는 작은따옴표는 문자열을 나타내는 기호로 인식되지 않는다. 대화형 인터프리터를 열고 다음과 같이 입력해 보자.

```
>>> food = "Python's favorite food is perl"
```

프롬프트에 `food`를 입력하면 변수에 저장된 문자열이 그대로 출력된다.

```
>>> food
"Python's favorite food is perl"
```

만약 문자열을 큰따옴표가 아닌 작은따옴표로 둘러싸면 'Python'이 문자열로 인식되어 구문 오류(SyntaxError)가 발생한다.

```
>>> food = 'Python's favorite food is perl'
  File "<stdin>", line 1
    food = 'Python's favorite food is perl'
                   ^
SyntaxError: invalid syntax
```

2. 문자열에 큰따옴표 포함하기

```
"Python is very easy." he says.
```

큰따옴표가 포함된 문자열은 작은따옴표로 둘러싸면 된다. 다음과 같이 입력해 보자.

```
>>> say = '"Python is very easy." he says.'
```

작은따옴표 안에 사용된 큰따옴표는 문자열을 만드는 기호로 인식되지 않는다.

3. 역슬래시를 사용해서 작은따옴표와 큰따옴표를 문자열에 포함하기

```
>>> food = 'Python\'s favorite food is perl'
>>> say = "\"Python is very easy.\" he says."
```

작은따옴표나 큰따옴표를 문자열에 포함시키는 또 다른 방법은 역슬래시(`\`)를 사용하는 것이다. 역슬래시를 작은따옴표나 큰따옴표 앞에 삽입하면 역슬래시 뒤의 작은따옴표나 큰따옴표는 문자열을 둘러싸는 기호가 아니라 문자 그 자체를 뜻한다.

어떤 방법을 사용할지는 자유롭게 선택하면 된다. 대화형 인터프리터를 실행한 후 위 예문을 직접 작성해 보자.

여러 줄인 문자열을 변수에 대입하고 싶을 때

다음과 같은 여러 줄의 문자열을 변수에 대입하려면 어떻게 해야 할까?

```
Life is too short
You need python
```

1. 줄을 바꾸기 위한 이스케이프 코드 `\n` 삽입하기

```
>>> multiline = "Life is too short\nYou need python"
```

줄바꿈 문자인 `\n`을 삽입하는 방법이 있지만, 읽기가 불편하고 줄이 길어지는 단점이 있다.

2. 연속된 작은따옴표 3개 또는 큰따옴표 3개 사용하기

1번 방법의 단점을 극복하기 위해 작은따옴표 3개(''') 또는 큰따옴표 3개(""")를 사용한다.

```
>>> multiline='''
... Life is too short
... You need python
... '''
```

작은따옴표 3개를 사용한 경우

```
>>> multiline="""
... Life is too short
... You need python
... """
```

큰따옴표 3개를 사용한 경우

`print(multiline)`을 입력하면 어떻게 출력되는지 확인해 보자.

```
>>> print(multiline)
Life is too short
You need python
```

두 경우 모두 결과는 동일하다. 문자열이 여러 줄인 경우, 이스케이프 코드를 쓰는 것보다 따옴표 3개를 사용하는 것이 훨씬 깔끔하다.

점프 투 파이썬

이스케이프 코드란?

문자열 예제에서 여러 줄의 문장을 처리할 때 역슬래시 문자와 소문자 n을 조합한 `\n` 이스케이프 코드를 사용했다. 이스케이프(escape) 코드란 프로그래밍할 때 사용할 수 있도록 미리 정의해 둔 '문자 조합'이다. 주로 출력물을 보기 좋게 정렬하는 용도로 사용한다. 몇 가지 이스케이프 코드를 정리하면 다음과 같다.

코드	설명
`\n`	문자열 안에서 줄을 바꿀 때 사용
`\t`	문자열 사이에 탭 간격을 줄 때 사용
`\\`	`\`를 그대로 표현할 때 사용
`\'`	작은따옴표(')를 그대로 표현할 때 사용
`\"`	큰따옴표(")를 그대로 표현할 때 사용
`\r`	캐리지 리턴(줄 바꿈 문자, 커서를 현재 줄의 가장 앞으로 이동)
`\f`	폼 피드(줄 바꿈 문자, 커서를 현재 줄의 다음 줄로 이동)
`\a`	벨 소리(출력할 때 PC 스피커에서 '삑' 소리가 난다)
`\b`	백 스페이스
`\000`	널 문자

이 중에서 활용 빈도가 높은 것은 `\n`, `\t`, `\\`, `\'`, `\"`이다. 나머지는 잘 사용하지 않는다.

문자열 연산하기

파이썬에서는 문자열을 더하거나 곱할 수 있다. 다른 언어에서는 쉽게 찾아볼 수 없는 재미있는 기능이다.

문자열 더해서 연결하기

```
>>> head = "Python"
>>> tail = " is fun!"
>>> head + tail
'Python is fun!'
```

"Python"이라는 head 변수와 " is fun!"이라는 tail 변수를 더한 결과는 'Python is fun!'이다. 즉, head와 tail 변수가 +에 의해 합쳐진 것이다. 직접 실행해 보자.

문자열 곱하기

```
>>> a = "python"
>>> a * 2
'pythonpython'
```

`a * 2`는 a를 2번 반복하라는 뜻이다. 즉, `*`는 문자열의 반복을 뜻한다.

문자열 곱하기 응용하기

문자열 곱하기를 응용해 보자. 다음 소스를 IDLE 에디터를 열어 작성해 보자.

```
# multistring.py

print("=" * 50)
print("My Program")
print("=" * 50)
```

입력한 소스는 `C:\doit` 디렉터리에 'multistring.py'라는 파일 이름으로 저장하자.

프로그램을 실행해 보자. [실행 창 열기(`윈도우 + R`) → 'cmd' 입력 → `Enter` 입력]으로 명령 프롬프트 창을 열고 다음을 따라 해 보자.

```
C:\Users>cd C:\doit
C:\doit>python multistring.py
==========================================
My Program
==========================================
```

이런 식의 표현은 앞으로 자주 사용하게 될 것이다. 프로그램 실행 시 출력 화면의 제일 위쪽에 프로그램 제목을 이와 같이 표시하면 보기 좋다.

문자열 길이 구하기

문자열의 길이는 len 함수를 사용하면 구할 수 있다. len 함수는 print 함수처럼 파이썬의 기본 내장 함수로, 별다른 설정 없이 바로 사용할 수 있다.

> *문자열의 길이에는 공백 문자도 포함된다.*

```
>>> a = "Life is too short"
>>> len(a)
17
```

문자열 인덱싱과 슬라이싱

인덱싱(indexing)은 무엇인가를 '가리킨다', 슬라이싱(slicing)은 무엇인가를 '잘라 낸다'라는 의미다. 이런 의미를 생각하면서 다음 내용을 살펴보자.

문자열 인덱싱

```
>>> a = "Life is too short, You need Python"
```

변수 a에 저장한 문자열의 각 문자마다 번호를 매겨 보면 다음과 같다.

L	i	f	e		i	s		t	o	o		s	h	o	r	t	,		Y	o	u		n	e	e	d		P	y	t	h	o	n
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33

"Life is too short, You need Python" 문자열에서 L은 첫 번째 자리를 뜻하는 숫자 0, i는 1 이런 식으로 계속 번호를 붙인 것이다. 중간에 있는 short의 s는 12가 된다.

이제 다음 예를 실행해 보자.

```
>>> a = "Life is too short, You need Python"
>>> a[3]
'e'
```

a[3]이 뜻하는 것은 a라는 문자열의 네 번째 문자 e다. a[3]에서 숫자 3이 왜 네 번째 문자를 뜻하는지 의아할 수 있다. 다음과 같이 생각하면 쉽다.

"파이썬은 0부터 숫자를 센다."

따라서 파이썬은 위 문자열을 다음과 같이 바라본다.

```
a[0]:'L', a[1]:'i', a[2]:'f', a[3]:'e', a[4]:' ', ...
```

0부터 숫자를 센다는 것이 처음에는 익숙하지 않겠지만, 계속 사용하다 보면 자연스러워진다. a[번호]는 문자열 안의 특정한 값을 뽑아 내는 역할을 한다. 이러한 작업을 '인덱싱'이라고 한다.

문자열 인덱싱 활용하기

인덱싱의 예를 몇 가지 더 살펴보자.

```
>>> a = "Life is too short, You need Python"
>>> a[0]
'L'
>>> a[12]
's'
>>> a[-1]
'n'
```

앞의 a[0]과 a[12]는 쉽게 이해할 수 있는데, 마지막의 a[-1]이 뜻하는 것은 뭘까? 문자열을 뒤에서부터 읽기 위해 -(빼기) 기호를 붙인 것이다. 즉, a[-1]은 뒤에서부터 세어 첫 번째가 되는 문자다. a의 값은 "Life is too short, You need Python" 문자열이므로 뒤에서부터 첫 번째 문자는 가장 마지막 문자 'n'이다.

뒤에서부터 첫 번째 문자를 표시할 때도 0부터 세어 'a[-0]이라고 해야 하지 않을까?'라는 의문이 들 수 있다. 하지만 0과 -0은 똑같은 것이기 때문에 a[-0]은 a[0]과 똑같은 값을 보여 준다.

```
>>> a[-0]
'L'
```

계속해서 몇 가지 예를 더 살펴보자.

```
>>> a[-2]
'o'
>>> a[-5]
'y'
```

첫 번째 예는 뒤에서부터 두 번째 문자, 두 번째 예는 뒤에서부터 다섯 번째 문자를 가리킨다.

문자열 슬라이싱

"Life is too short, You need Python" 문자열에서 한 문자만이 아니라 'Life' 또는 'You'와 같은 단어를 뽑아 내는 방법은 없을까?

다음과 같이 하면 된다.

```
>>> a = "Life is too short, You need Python"
>>> b = a[0] + a[1] + a[2] + a[3]
>>> b
'Life'
```

위와 같이 단순하게 접근할 수도 있지만 파이썬에서는 더 좋은 방법을 제공한다. 바로 슬라이싱(slicing) 기법이다. 위 예는 슬라이싱 기법으로 다음과 같이 간단하게 처리할 수 있다.

> *인덱싱 기법과 슬라이싱 기법은 뒤에서 배울 자료형인 리스트나 튜플에서도 사용할 수 있다.*

```
>>> a = "Life is too short, You need Python"
>>> a[0:4]
'Life'
```

a[0:4]는 a 문자열, 즉 "Life is too short, You need Python" 문자열에서 자리 번호 0부터 4까지의 문자를 뽑아 낸다는 뜻이다.

하지만 'a[0]은 L, a[1]은 i, a[2]는 f, a[3]은 e이므로 a[0:3]으로도 Life라는 단어를 뽑아 낼 수 있지 않을까?'라는 의문이 생길 것이다. 다음 예로 확인해 보자.

```
>>> a[0:3]
'Lif'
```

이렇게 되는 이유는 슬라이싱 기법으로 a[시작_번호:끝_번호]를 지정할 때 끝 번호에 해당하는 문자는 포함하지 않기 때문이다. 즉, a[0:3]을 수식으로 나타내면 다음과 같다.

```
0 ≤ a < 3
```

이 수식을 만족하는 것은 a[0], a[1], a[2]다. 따라서 a[0:3]은 'Lif', a[0:4]는 'Life'가 되는 것이다. 이 부분이 문자열 연산에서 가장 혼동하기 쉬운 부분이므로 02장의 마지막 부분에 있는 되새김 문제를 많이 풀어 보면서 익히기 바란다.

문자열을 슬라이싱하는 방법

슬라이싱의 예를 좀 더 살펴보자.

```
>>> a[0:5]
'Life '
```

위 예는 a[0] + a[1] + a[2] + a[3] + a[4]와 동일하다. a[4]는 공백 문자이기 때문에 'Life'가 아닌 'Life '가 출력된다. 공백 문자 역시 L, i, f, e 와 같은 문자와 동일하게 취급된다. 'Life'와 'Life '는 완전히 다른 문자열이다.

슬라이싱할 때 항상 시작 번호가 0일 필요는 없다.

```
>>> a[0:2]
'Li'
>>> a[5:7]
'is'
>>> a[12:17]
'short'
```

a[시작_번호:끝_번호]에서 끝 번호 부분을 생략하면 시작 번호부터 그 문자열의 끝까지 뽑아 낸다.

```
>>> a[19:]
'You need Python'
```

a[시작_번호:끝_번호]에서 시작 번호를 생략하면 문자열의 처음부터 끝 번호까지 뽑아 낸다.

```
>>> a[:17]
'Life is too short'
```

a[시작_번호:끝_번호]에서 시작 번호와 끝 번호를 생략하면 문자열의 처음부터 끝까지 뽑아 낸다.

```
>>> a[:]
'Life is too short, You need Python'
```

슬라이싱에서도 인덱싱과 마찬가지로 -(빼기) 기호를 사용할 수 있다.

```
>>> a[19:-7]
'You need'
```

a[19:-7]은 a[19]에서 a[-8]까지를 의미한다. 이때에도 a[-7]은 포함하지 않는다.

슬라이싱으로 문자열 나누기

다음은 자주 사용하는 슬라이싱 기법 중 하나다.

```
>>> a = "20230331Rainy"
>>> date = a[:8]
>>> weather = a[8:]
>>> date
'20230331'
>>> weather
'Rainy'
```

위 예는 문자열 a를 두 부분으로 나누는 기법이다. 숫자 8을 기준으로 문자열 a를 양쪽으로 한 번씩 슬라이싱했다. a[:8]은 a[8]을 포함하지 않고 a[8:]은 a[8]을 포함하기 때문에 8을 기준으로 두 부분으로 나눌 수 있다. "20230331Rainy" 문자열을 날짜를 나타내는 부분인 '20230331'과 날씨를 나타내는 부분인 'Rainy'로 나누었다.

"20230331Rainy"를 연도인 2023, 월과 일을 나타내는 0331, 날씨를 나타내는 Rainy까지 세 부분으로 나누는 방법은 다음과 같다.

```
>>> a = "20230331Rainy"
>>> year = a[:4]
>>> day = a[4:8]
>>> weather = a[8:]
>>> year
'2023'
>>> day
'0331'
>>> weather
'Rainy'
```

위 예는 숫자 4와 8로 "20230331Rainy" 문자열을 세 부분으로 나누는 방법을 보여 준다.

인덱싱과 슬라이싱은 프로그래밍할 때 자주 사용하는 기법이므로 반복해서 연습해 두자.

점프 투 파이썬

Pithon 문자열을 Python으로 바꾸려면?

Pithon 문자열을 Python으로 바꾸려면 어떻게 해야 할까? 제일 먼저 떠오르는 생각은 다음과 같을 것이다.

```
>>> a = "Pithon"
>>> a[1]
'i'
>>> a[1] = 'y'
```

즉, a 변수에 "Pithon" 문자열을 대입하고 a[1]의 값이 i이므로 a[1]을 y로 바꾸어 준다는 생각이다. 하지만 오류가 발생한다. 문자열의 요솟값은 바꿀 수 없는 값이기 때문이다(그래서 문자열을 '변경 불가능한(immutable) 자료형'이라고도 부른다).

하지만 슬라이싱 기법을 사용하면 Pithon 문자열을 사용해 Python 문자열을 만들 수 있다. 다음 예를 살펴보자.

```
>>> a = "Pithon"
>>> a[:1]
'P'
>>> a[2:]
'thon'
>>> a[:1] + 'y' + a[2:]
'Python'
```

슬라이싱을 사용하면 "Pithon" 문자열을 'P' 부분과 'thon' 부분으로 나눌 수 있고, 그 사이에 'y' 문자를 추가하면 'Python'이라는 새로운 문자열을 만들 수 있다.

문자열 포매팅이란?

문자열에서 또 하나 알아야 할 것으로 '문자열 포매팅(string formatting)'이 있다. 다음과 같은 문자열을 출력하는 프로그램을 작성했다고 가정해 보자.

```
"현재 온도는 18도입니다."
```

시간이 지나서 20도가 되면 다음 문장을 출력한다.

```
"현재 온도는 20도입니다."
```

두 문자열은 모두 같은데 20이라는 숫자와 18이라는 숫자만 다르다. 이렇게 문자열 안의 특정한 값을 바꿔야 할 경우가 있을 때 이것을 가능하게 해 주는 것이 바로 문자열 포매팅이다.

쉽게 말해 문자열 포매팅은 문자열 안에 어떤 값을 삽입하는 방법이다. 다음 예를 직접 실행해 보면서 사용법을 알아보자.

문자열 포매팅 따라 하기

1. 숫자 바로 대입

```
>>> "I eat %d apples." % 3
'I eat 3 apples.'
```

위 예제는 문자열 안에 정수 3을 삽입하는 방법을 보여 준다. 문자열 안의 숫자를 넣고 싶은 자리에 `%d` 문자를 넣어 주고 삽입할 숫자 3은 가장 뒤에 있는 `%` 문자 다음에 써 넣었다. 여기에서 `%d` 는 '문자열 포맷 코드'라고 부른다.

2. 문자열 바로 대입

문자열 안에 꼭 숫자만 넣으라는 법은 없다. 이번에는 숫자 대신 문자열을 넣어 보자.

```
>>> "I eat %s apples." % "five"
'I eat five apples.'
```

문자열 안에 또 다른 문자열을 삽입하기 위해 앞에서 사용한 문자열 포맷 코드 `%d`가 아닌 `%s`를 썼다. 숫자를 넣기 위해서는 `%d`, 문자열을 넣기 위해서는 `%s`를 써야 한다.

> *앞에서 배운 것처럼 문자열을 대입할 때는 반드시 큰따옴표나 작은따옴표를 써야 한다.*

3. 숫자 값을 나타내는 변수로 대입

```
>>> number = 3
>>> "I eat %d apples." % number
'I eat 3 apples.'
```

1번처럼 숫자를 바로 대입하든, 위 예제처럼 숫자 값을 나타내는 변수를 대입하든 결과는 같다.

4. 2개 이상의 값 넣기

문자열 안에 여러 개의 값을 넣고 싶을 때는 어떻게 해야 할까?

```
>>> number = 10
>>> day = "three"
>>> "I ate %d apples. so I was sick for %s days." % (number, day)
'I ate 10 apples. so I was sick for three days.'
```

2개 이상의 값을 넣으려면 마지막 `%` 다음 괄호 안에 쉼표(,)로 구분하여 각각의 값을 넣어 주면 된다.

문자열 포맷 코드

문자열 포매팅 예제에서는 대입해 넣는 자료형으로 정수와 문자열을 사용했지만, 이 밖에도 다양한 것을 대입할 수 있다. 문자열 포맷 코드의 종류는 다음과 같다.

코드	설명
%s	문자열(String)
%c	문자 1개(character)
%d	정수(Integer)
%f	부동소수(floating-point)
%o	8진수
%x	16진수
%%	Literal % (문자 `%` 자체)

여기에서 재미있는 것은 `%s` 포맷 코드인데, 이 코드에는 어떤 형태의 값이든 변환해 넣을 수 있다. 예를 통해 확인해 보자.

```
>>> "I have %s apples" % 3
'I have 3 apples'
>>> "rate is %s" % 3.234
'rate is 3.234'
```

3을 문자열 안에 삽입하려면 `%d` 를 사용해야 하고 3.234를 삽입하려면 `%f` 를 사용해야 한다. 하지만 `%s` 를 사용하면 `%s` 는 자동으로 `%` 뒤에 있는 3이나 3.234와 같은 값을 문자열로 바꾸어 대입한다.

점프 투 파이썬

포매팅 연산자 `%d` 와 `%` 를 같이 쓸 때는 `%%` 를 쓴다

```
>>> "Error is %d%." % 98
```

결괏값으로 당연히 "Error is 98%."가 출력될 것이라고 예상하겠지만, 파이썬은 '형식이 불완전하다'라는 오류 메시지를 보여 준다.

```
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: incomplete format
```

그 이유는 '문자열 포맷 코드인 `%d` 와 `%` 가 같은 문자열 안에 존재하는 경우, `%` 를 나타내려면 반드시 `%%` 를 써야 한다'라는 법칙이 있기 때문이다. 이 점은 꼭 기억해 두어야 한다. 하지만 문자열 안에 `%d` 와 같은 포매팅 연산자가 없으면 `%` 는 홀로 쓰여도 상관없다. 따라서 위 예를 제대로 실행하려면 다음과 같이 작성해야 한다.

```
>>> "Error is %d%%." % 98
'Error is 98%.'
```

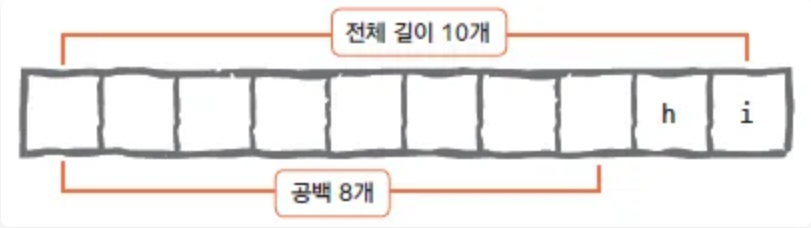
포맷 코드와 숫자 함께 사용하기

`%d` , `%s` 등과 같은 포맷 코드는 문자열 안에 어떤 값을 삽입할 때 사용한다. 하지만 포맷 코드를 숫자와 함께 사용하면 더 유용하다. 다음 예를 따라해 보자.

1. 정렬과 공백

```
>>> "%10s" % "hi"
'        hi'
```

`%10s` 는 전체 길이가 10개인 문자열 공간에서 대입되는 값을 오른쪽으로 정렬하고 그 앞의 나머지는 공백으로 남겨 두라는 의미다.



반대쪽인 왼쪽 정렬은 `%-10s` 가 된다.

```
>>> "%-10sjane." % 'hi'
'hi        jane.'
```

hi를 왼쪽으로 정렬하고 나머지는 공백으로 채웠다.



2. 소수점 표현하기

```
>>> "%0.4f" % 3.42134234
'3.4213'
```

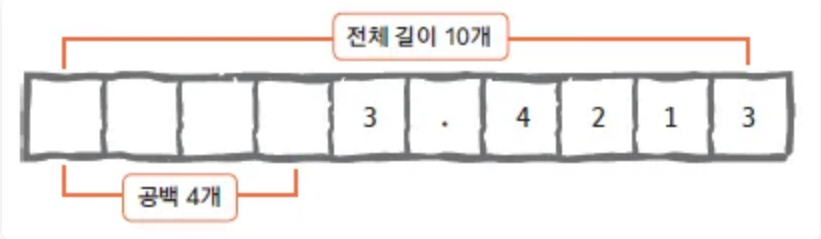
3.42134234를 소수점 네 번째 자리까지만 나타내고 싶은 경우에는 위와 같이 작성한다. `%0.4f`에서 '.'는 소수점 포인트, 그 뒤의 숫자 4는 소수점 뒤에 나올 숫자의 개수다. 소수점 포인트 앞의 숫자는 문자열의 전체 길이를 의미하는데, `%0.4f`에서 사용한 숫자 0은 길이에 상관하지 않겠다는 의미다.

> *`%0.4f` 는 0을 생략하여 `%.4f` 처럼 사용하기도 한다.*

다음 예를 살펴보자.

```
>>> "%10.4f" % 3.42134234
'    3.4213'
```

위는 숫자 3.42134234를 소수점 네 번째 자리까지만 표시하고 전체 길이가 10개인 문자열 공간에서 오른쪽으로 정렬하는 예를 보여 준다.



format 함수를 사용한 포매팅

문자열의 format 함수를 사용하면 좀 더 발전된 스타일로 문자열 포맷을 지정할 수 있다. 앞에서 살펴본 문자열 포매팅 예제를 format 함수를 사용해서 바꾸면 다음과 같다.

숫자 바로 대입하기

```
>>> "I eat {0} apples".format(3)
'I eat 3 apples'
```

"I eat {0} apples" 문자열 중 {0} 부분이 숫자 3으로 바뀌었다.

문자열 바로 대입하기

```
>>> "I eat {0} apples".format("five")
'I eat five apples'
```

문자열의 {0} 항목이 'five'라는 문자열로 바뀌었다.

숫자 값을 가진 변수로 대입하기

```
>>> number = 3
>>> "I eat {0} apples".format(number)
'I eat 3 apples'
```

문자열의 {0} 항목이 number 변수의 값인 3으로 바뀌었다.

2개 이상의 값 넣기

```
>>> number = 10
>>> day = "three"
>>> "I ate {0} apples. so I was sick for {1} days.".format(number, day)
'I ate 10 apples. so I was sick for three days.'
```

2개 이상의 값을 넣을 경우, 문자열의 `{0}`, `{1}`과 같은 인덱스 항목이 format 함수의 입력값으로 순서에 맞게 바뀐다. 위 예에서 `{0}`은 format 함수의 첫 번째 입력값인 number, `{1}`은 format 함수의 두 번째 입력값인 day로 바뀐다.

이름으로 넣기

```
>>> "I ate {number} apples. so I was sick for {day} days.".format(number=10, day=3)
'I ate 10 apples. so I was sick for 3 days.'
```

`{0}`, `{1}`과 같은 인덱스 항목 대신 더 편리한 `{name}` 형태를 사용하는 방법도 있다. `{name}` 형태를 사용할 경우, format 함수에는 반드시 `name=value`와 같은 형태의 입력값이 있어야 한다. 위 예는 문자열의 `{number}`, `{day}`가 format 함수의 입력값인 `number=10, day=3` 값으로 각각 바뀌는 것을 보여 준다.

인덱스와 이름을 혼용해서 넣기

```
>>> "I ate {0} apples. so I was sick for {day} days.".format(10, day=3)
'I ate 10 apples. so I was sick for 3 days.'
```

인덱스 항목과 `name=value` 형태를 혼용하는 것도 가능하다.

왼쪽 정렬

```
>>> "{0:<10}".format("hi")
'hi        '
```

`:<10` 표현식을 사용하면 치환되는 문자열을 왼쪽으로 정렬하고 문자열의 총 자릿수를 10으로 맞출 수 있다.

오른쪽 정렬

```
>>> "{0:>10}".format("hi")
'        hi'
```

오른쪽 정렬은 `:<` 대신 `:>`을 사용하면 된다. 화살표의 방향을 생각하면 어느 쪽으로 정렬되는지 바로 알 수 있다.

가운데 정렬

```
>>> "{0:^10}".format("hi")
'    hi    '
```

`:^`를 사용하면 가운데 정렬도 가능하다.

공백 채우기

```
>>> "{0:=^10}".format("hi")
'════hi════'
>>> "{0:!<10}".format("hi")
'hi!!!!!!!!'
```

정렬할 때 공백 문자 대신 지정한 문자 값으로 채워 넣을 수도 있다. 채워 넣을 문자 값은 정렬 문자 `<`, `>`, `^` 바로 앞에 넣어야 한다. 위 예에서 첫 번째 예제는 가운데(`^`)로 정렬하고 빈 공간을 `=`로 채웠고, 두 번째 예제는 왼쪽(`<`)으로 정렬하고 빈 공간을 `!`로 채웠다.

소수점 표현하기

```
>>> y = 3.42134234
>>> "{0:0.4f}".format(y)
'3.4213'
```

위는 format 함수를 사용해 소수점을 4자리까지만 표현하는 방법을 보여 준다. 앞에서 살펴보았던 표현식 0.4f를 그대로 사용한 것을 알 수 있다.

```
>>> "{0:10.4f}".format(y)
'    3.4213'
```

위와 같이 자릿수를 10으로 맞출 수도 있다. 이 또한 앞에서 살펴본 10.4f의 표현식을 그대로 사용한 것을 알 수 있다.

{ 또는 } 문자 표현하기

```
>>> "{{ and }}".format()
'{ and }'
```

format 함수를 사용해 문자열을 포매팅할 경우, `{}` 와 같은 중괄호 문자를 포매팅 문자가 아닌 문자 그대로 사용하고 싶은 경우에는 위 예의 `{{}}` 처럼 2개를 연속해서 사용하면 된다.

f 문자열 포매팅

파이썬 3.6 버전부터는 f 문자열 포매팅 기능을 사용할 수 있다. 파이썬 3.6 미만 버전에서는 사용할 수 없는 기능이므로 주의해야 한다.

다음과 같이 문자열 앞에 f 접두사를 붙이면 f 문자열 포매팅 기능을 사용할 수 있다.

```
>>> name = '홍길동'
>>> age = 30
>>> f'나의 이름은 {name}입니다. 나이는 {age}입니다.'
'나의 이름은 홍길동입니다. 나이는 30입니다.'
```

f 문자열 포매팅은 위와 같이 name, age와 같은 변숫값을 생성한 후에 그 값을 참조할 수 있다. 또한 f 문자열 포매팅은 표현식을 지원하기 때문에 다음과 같은 것도 가능하다.

> *표현식이란 중괄호 안의 변수를 계산식과 함께 사용하는 것이다.*

```
>>> age = 30
>>> f'나는 내년이면 {age + 1}살이 된다.'
'나는 내년이면 31살이 된다.'
```

딕셔너리는 f 문자열 포매팅에서 다음과 같이 사용할 수 있다.

```
>>> d = {'name':'홍길동', 'age':30}
>>> f'나의 이름은 {d["name"]}입니다. 나이는 {d["age"]}입니다.'
'나의 이름은 홍길동입니다. 나이는 30입니다.'
```

> *딕셔너리는 Key와 Value라는 것을 한 쌍으로 가지는 자료형이다. 02-5에서 자세히 알아본다.*

정렬은 다음과 같이 할 수 있다.

```
>>> f'{"hi":<10}'  # 왼쪽 정렬
'hi        '
>>> f'{"hi":>10}'  # 오른쪽 정렬
'        hi'
>>> f'{"hi":^10}'  # 가운데 정렬
'    hi    '
```

공백 채우기는 다음과 같이 할 수 있다.

```
>>> f'{"hi":=^10}'  # 가운데 정렬하고 '=' 문자로 공백 채우기
'====hi===='
>>> f'{"hi":!<10}'  # 왼쪽 정렬하고 '!' 문자로 공백 채우기
'hi!!!!!!!!'
```

소수점은 다음과 같이 표현할 수 있다.

```
>>> y = 3.42134234
>>> f'{y:0.4f}'  # 소수점 4자리까지만 표현
'3.4213'
>>> f'{y:10.4f}'  # 소수점 4자리까지 표현하고 총 자리수를 10으로 맞춤
'    3.4213'
```

f 문자열에서 `{}`를 문자 그대로 표시하려면 다음과 같이 2개를 동시에 사용해야 한다.

```
>>> f'{{ and }}'
'{ and }'
```

점프 투 파이썬

f 문자열을 사용하여 금액에 콤마(,) 삽입하기

다음 문장을 살펴보자.

> *"난 1500000원이 필요해"*

이 문장에서 1500000(백오십만)원에 해당하는 금액 부분은 콤마가 없어서 읽기가 쉽지 않다. 이때, f 문자열을 활용하면 숫자에 자동으로 **콤마(,)**를 삽입하여 가독성을 높일 수 있다.

```
>>> f"난 {1500000:,}원이 필요해"
'난 1,500,000원이 필요해'
```

출력 결과를 보면 1,500,000처럼 숫자 사이에 콤마가 삽입되어 금액을 더 쉽게 읽을 수 있다.

지금까지는 문자열을 가지고 할 수 있는 기본적인 것에 대해 알아보았다. 이제부터는 문자열을 좀 더 자유자재로 다루기 위해 공부해야 할 것을 설명한다.

문자열 관련 함수들

문자열 자료형은 자체적으로 함수를 가지고 있다. 이들 함수를 다른 말로 '문자열 내장 함수'라고 한다. 이 내장 함수를 사용하려면 문자열 변수 이름 뒤에 '.'를 붙인 후 함수 이름을 써 주면 된다. 이제 문자열의 내장 함수에 대해서 알아보자.

문자 개수 세기 - count

```
>>> a = "hobby"
>>> a.count('b')
2
```

count 함수로 문자열 중 문자 b의 개수를 반환했다.

위치 알려 주기 1 - find

```
>>> a = "Python is the best choice"
>>> a.find('b')
14
>>> a.find('k')
-1
```

find 함수로 문자열 중 문자 b가 처음으로 나온 위치를 반환했다. 만약 찾는 문자나 문자열이 존재하지 않는다면 -1을 반환한다.

> *파이썬은 숫자를 0부터 세기 때문에 b의 위치는 15가 아닌 14가 된다.*

위치 알려 주기 2 - index

```
>>> a = "Life is too short"
>>> a.index('t')
8
>>> a.index('k')
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: substring not found
```

index 함수로 문자열 중 문자 t가 맨 처음으로 나온 위치를 반환했다. 만약 찾는 문자나 문자열이 존재하지 않는다면 오류가 발생한다. 앞의 find 함수와 다른 점은 문자열 안에 존재하지 않는 문자를 찾으면 오류가 발생한다는 것이다.

문자열 삽입 - join

```
>>> ",".join('abcd')
'a,b,c,d'
```

join 함수로 abcd 문자열의 각각의 문자 사이에 ','를 삽입했다.

join 함수는 문자열뿐만 아니라 앞으로 배울 리스트나 튜플도 입력으로 사용할 수 있다(리스트와 튜플은 곧 배울 내용이므로 여기에서는 잠시 눈으로만 살펴보자). join 함수의 입력으로 리스트를 사용하는 예는 다음과 같다.

```
>>> ",".join(['a', 'b', 'c', 'd'])
'a,b,c,d'
```

소문자를 대문자로 바꾸기 - upper

```
>>> a = "hi"
>>> a.upper()
'HI'
```

upper 함수는 소문자를 대문자로 바꾸어 준다. 만약 문자열이 이미 대문자라면 아무런 변화도 일어나지 않는다.

대문자를 소문자로 바꾸기 - lower

```
>>> a = "HI"
>>> a.lower()
'hi'
```

lower 함수는 대문자를 소문자로 바꾸어 준다.

왼쪽 공백 지우기 - lstrip

```
>>> a = " hi "
>>> a.lstrip()
'hi '
```

lstrip 함수는 문자열 중 가장 왼쪽에 있는 한 칸 이상의 연속된 공백들을 모두 지운다. lstrip에서 l은 left를 의미한다.

오른쪽 공백 지우기 - rstrip

```
>>> a= " hi "
>>> a.rstrip()
' hi'
```

rstrip 함수는 문자열 중 가장 오른쪽에 있는 한 칸 이상의 연속된 공백을 모두 지운다. rstrip에서 r은 right를 의미한다.

양쪽 공백 지우기 - strip

```
>>> a = " hi "
>>> a.strip()
'hi'
```

strip 함수는 문자열 양쪽에 있는 한 칸 이상의 연속된 공백을 모두 지운다.

문자열 바꾸기 - replace

```
>>> a = "Life is too short"
>>> a.replace("Life", "Your leg")
'Your leg is too short'
```

replace 함수는 replace(바뀔_문자열, 바꿀_문자열)처럼 사용해서 문자열 안의 특정한 값을 다른 값으로 치환해 준다.

문자열 나누기 - split

```
>>> a = "Life is too short"
>>> a.split()
['Life', 'is', 'too', 'short']
>>> b = "a:b:c:d"
>>> b.split(':')
['a', 'b', 'c', 'd']
```

split 함수는 a.split()처럼 괄호 안에 아무 값도 넣어 주지 않으면 공백(`[Space]`, `[Tab]`, `[Enter]`)을 기준으로 문자열을 나누어 준다. 만약 b.split(':')처럼 괄호 안에 특정 값이 있을 경우에는 괄호 안의 값을 구분자로 해서 문자열을 나누어 준다. 이렇게 나눈 값은 리스트에 하나씩 들어간다. `['Life', 'is', 'too', 'short']`나 `['a', 'b', 'c', 'd']`가 리스트인데, 02-3에서 자세히 알아볼 것이므로 여기에서는 너무 신경 쓰지 않아도 된다.

문자열이 알파벳으로만 구성되어 있는지 확인하기 - isalpha

```
>>> s = "Python"
>>> s.isalpha()
True
>>> s = "Python3"
>>> s.isalpha()
False
>>> s = "Hello World"
>>> s.isalpha()
False
```

isalpha 함수는 문자열이 문자(letters)로만 이루어져 있는지 검사한다. 문자열의 모든 문자가 문자이면 `True`를 반환하며, 공백이나 숫자, 특수 문자가 포함되어 있으면 `False`를 반환한다.

"Python3"의 경우 숫자가 포함되어 있어서 `False`를 반환하고, "Hello World"의 경우 공백 문자가 포함되어 있어 `False`를 반환한다.

문자열이 숫자로만 구성되어 있는지 확인하기 - isdigit

```
>>> s = "12345"
>>> s.isdigit()
True
>>> s = "1234a"
>>> s.isdigit()
False
>>> s = "12 34"
>>> s.isdigit()
False
```

`isdigit` 함수는 문자열이 숫자 문자로만 이루어져 있는지를 검사한다. 문자열의 모든 문자가 숫자 문자일 경우 `True`를 반환하고, 하나라도 숫자 문자가 아닌 문자가 포함되어 있으면 `False`를 반환한다.

예를 들어 `"1234a"`는 알파벳 `a`가 포함되어 있어 `False`를 반환하며, `"12 34"`는 숫자 사이에 공백이 있으므로 `False`를 반환한다.

문자열이 특정 문자(열)로 시작하는지 확인하기 - startswith

```
>>> s = "Life is too short"
>>> s.startswith("Life")
True
>>> s.startswith("short")
False
```

`startswith` 함수는 문자열이 지정한 문자(열)로 시작하는지를 검사한다. 지정한 문자열로 **시작하면** `True`를 반환하고, 그렇지 않으면 `False`를 반환한다.

문자열이 특정 문자(열)로 끝나는지 확인하기 - endswith

```
>>> s = "Life is too short"
>>> s.endswith("short")
True
>>> s.endswith("too")
False
```

`endswith` 함수는 문자열이 지정한 문자(열)로 끝나는지를 검사한다. 지정한 문자열로 **끝나면** `True`를 반환하고, 그렇지 않으면 `False`를 반환한다.

앞에서 소개한 문자열 관련 함수는 문자열 처리에서 사용 빈도가 매우 높고 유용하다. 이 밖에도 몇 가지가 더 있지만 자주 사용하지는 않는다.

점프 투 파이썬

착각하기 쉬운 문자열 함수

소문자를 대문자로 바꾸어 주는 다음의 예를 보자.

```
>>> a = 'hi'
>>> a.upper()
'HI'
```

이와 같이 실행한 후에 a 변수의 값은 'HI'로 변했을까? 아니면 'hi' 값을 유지할까? 다음과 같이 확인해 보자.

```
>>> a
'hi'
```

a.upper()를 수행하더라도 a 변수의 값은 변하지 않았다. 왜냐하면 a.upper()를 실행하면 upper 함수는 a 변수의 값 자체를 변경하는 것이 아니라 대문자로 바꾼 값을 반환하기 때문이다. 문자열은 이전에도 잠깐 언급했지만 자체의 값을 변경할 수 없는 immutable 자료형이다. 따라서 a 값을 'HI' 로 바꾸고 싶다면 다음과 같이 대입문을 사용해야 한다.

```
>>> a = a.upper()
>>> a
'HI'
```

upper 뿐만 아니라 lower, join, lstrip, rstrip, strip, replace, split 함수는 모두 이와 같은 규칙이 적용되어 문자열 자체의 값이 변경되는 것이 아니라 변경된 값을 반환한다는 사실에 주의하자.

02-3 리스트 자료형

지금까지 우리는 숫자와 문자열에 대해서 알아보았다. 하지만 숫자와 문자열만으로 프로그래밍을 하기에는 부족한 점이 많다. 예를 들어 1부터 10까지의 숫자 중 홀수의 모음인 1, 3, 5, 7, 9의 집합을 생각해 보자. 이런 숫자의 모음을 숫자나 문자열로 표현하기는 어렵다. 파이썬에는 이러한 불편함을 해소할 수 있는 자료형이 존재한다. 이것이 바로 지금부터 공부할 리스트(list)이다.

- 리스트는 어떻게 만들고 사용할까?
- 리스트의 인덱싱과 슬라이싱
  - 리스트의 인덱싱
  - 리스트의 슬라이싱
- 리스트 연산하기
  - 리스트 더하기(+)
  - 리스트 반복하기(*)
  - 리스트 길이 구하기
- 리스트의 수정과 삭제
  - 리스트의 값 수정하기
  - del로 리스트 요소 삭제하기
- 리스트 관련 함수
  - 리스트에 요소 추가하기 - append
  - 리스트 정렬 - sort
  - 리스트 뒤집기 - reverse
  - 인덱스 반환 - index
  - 리스트에 요소 삽입 - insert
  - 리스트 요소 제거 - remove
  - 리스트 요소 꺼내기 - pop
  - 리스트에 포함된 요소 x의 개수 세기 - count
  - 리스트 확장 - extend

리스트는 어떻게 만들고 사용할까?

리스트를 사용하면 1, 3, 5, 7, 9의 숫자 모음을 다음과 같이 간단하게 표현할 수 있다.

```
>>> odd = [1, 3, 5, 7, 9]
```

리스트를 만들 때는 위에서 보는 것과 같이 대괄호([])로 감싸 주고 각 요솟값은 쉼표(,)로 구분해 준다.

```
리스트명 = [요소1, 요소2, 요소3, ...]
```

여러 가지 리스트의 생김새는 다음과 같다.

```
>>> a = []
>>> b = [1, 2, 3]
>>> c = ['Life', 'is', 'too', 'short']
>>> d = [1, 2, 'Life', 'is']
>>> e = [1, 2, ['Life', 'is']]
```

> *비어 있는 리스트는 `a = list()` 로 생성할 수 있다.*

리스트는 a처럼 아무것도 포함하지 않아 비어 있는 리스트([])일 수도 있고, b처럼 숫자를 요솟값으로 가질 수도 있으며, c처럼 문자열을 요솟값으로 가질 수도 있다. 또한 d처럼 숫자와 문자열을 함께 요솟값으로 가질 수도 있고, e처럼 리스트 자체를 요솟값으로 가질 수도 있다. 즉, 리스트 안에는 어떠한 자료형도 포함할 수 있다.



리스트의 인덱싱과 슬라이싱

리스트도 문자열처럼 인덱싱과 슬라이싱이 가능하다. 직접 예를 실행해 보면서 리스트의 기본 구조를 이해해 보자. 대화형 인터프리터로 따라 하면서 확실하게 이해하자.

리스트의 인덱싱

리스트 역시 문자열처럼 인덱싱을 적용할 수 있다. 먼저 a 변수에 [1, 2, 3] 값을 설정한다.

```
>>> a = [1, 2, 3]
>>> a
[1, 2, 3]
```

a[0]은 리스트 a의 첫 번째 요솟값을 말한다.

```
>>> a[0]
1
```

다음 예는 리스트의 첫 번째 요소인 a[0]과 세 번째 요소인 a[2]의 값을 더한 것이다.

```
>>> a[0] + a[2]
4
```

이것은 1 + 3으로 해석되어 값 4를 출력한다.

문자열을 공부할 때 이미 살펴보았지만, 파이썬은 숫자를 0부터 세기 때문에 a[1]이 리스트 a의 첫 번째 요소가 아니라 a[0]이 리스트 a의 첫 번째 요소라는 것을 명심하자. a[-1]은 문자열에서와 마찬가지로 리스트 a의 마지막 요솟값을 말한다.

```
>>> a[-1]
3
```

이번에는 다음 예처럼 리스트 a를 숫자 1, 2, 3과 또 다른 리스트인 ['a', 'b', 'c']를 포함하도록 만들어 보자.

```
>>> a = [1, 2, 3, ['a', 'b', 'c']]
```

그리고 다음 예를 따라 해 보자.

```
>>> a[0]
1
>>> a[-1]
['a', 'b', 'c']
>>> a[3]
['a', 'b', 'c']
```

예상한 대로 a[-1]은 마지막 요솟값 ['a', 'b', 'c']를 나타낸다. a[3]은 리스트 a의 네 번째 요소를 나타내기 때문에 마지막 요소를 나타내는 a[-1]과 동일한 결괏값을 보여 준다.

그렇다면 리스트 a에 포함된 ['a', 'b', 'c'] 리스트에서 'a' 값을 인덱싱을 사용해 꺼낼 수 있는 방법은 없을까? 다음 예를 살펴보자.

```
>>> a[-1][0]
'a'
```

위와 같이 하면 'a'를 꺼낼 수 있다. a[-1]이 ['a', 'b', 'c'] 리스트라는 것은 이미 설명했다. 바로 이 리스트에서 첫 번째 요소를 불러오기 위해 [0]을 붙여 준 것이다.

다음 예도 마찬가지이므로 어렵지 않게 이해할 수 있을 것이다.

```
>>> a[-1][1]
'b'
>>> a[-1][2]
'c'
```

점프 투 파이썬

삼중 리스트에서 인덱싱하기

조금 복잡하지만, 리스트를 다음과 같이 작성할 수도 있다.

```
>>> a = [1, 2, ['a', 'b', ['Life', 'is']]]
```

리스트 a 안에 ['a', 'b', ['Life', 'is']] 리스트가 포함되어 있고 그 리스트 안에 다시 ['Life', 'is'] 리스트가 포함되어 있다. 즉, 삼중 구조의 리스트이다.

이 경우, 'Life' 문자열만 꺼내려면 다음과 같이 해야 한다.

```
>>> a[2][2][0]
'Life'
```

위 예는 리스트 a의 세 번째 요소인 리스트 ['a', 'b', ['Life', 'is']]에서 세 번째 요소인 리스트 ['Life', 'is']의 첫 번째 요소를 나타낸다.

리스트를 다중으로 중첩해서 사용하면 혼란스럽기 때문에 자주 사용하지는 않는다.

리스트의 슬라이싱

문자열과 마찬가지로 리스트에서도 슬라이싱 기법을 적용할 수 있다. 슬라이싱은 '잘라 낸다'라는 뜻이라고 했다. 리스트의 슬라이싱에 대해서 살펴보자.

```
>>> a = [1, 2, 3, 4, 5]
>>> a[0:2]
[1, 2]
```

앞의 예를 문자열에서 슬라이싱했던 예와 비교해 보자.

```
>>> a = "12345"
>>> a[0:2]
'12'
```

2가지가 완전히 동일하게 사용되었다. 문자열에서 했던 것과 사용법이 완전히 동일하다.

몇 가지 예를 더 살펴보자.

```
>>> a = [1, 2, 3, 4, 5]
>>> b = a[:2]
>>> c = a[2:]
>>> b
[1, 2]
>>> c
[3, 4, 5]
```

b 변수는 리스트 a의 첫 번째 요소부터 두 번째 요소인 a[1]까지 나타내는 리스트이다. 물론 a[2] 값인 3은 포함되지 않는다. c 변수는 리스트 a의 세 번째 요소부터 끝까지 나타내는 리스트이다.

점프 투 파이썬

중첩된 리스트에서 슬라이싱하기

리스트가 포함된 중첩 리스트 역시 슬라이싱 방법은 똑같이 적용된다.

```
>>> a = [1, 2, 3, ['a', 'b', 'c'], 4, 5]
>>> a[2:5]
[3, ['a', 'b', 'c'], 4]
>>> a[3][:2]
['a', 'b']
```

위 예에서 a[3]은 ['a', 'b', 'c']를 나타낸다. 따라서 a[3][:2]는 ['a', 'b', 'c']의 첫 번째 요소부터 세 번째 요소 직전까지의 값, 즉 ['a', 'b']를 나타내는 리스트가 된다.

리스트 연산하기

리스트 역시 +를 사용해서 더할 수 있고 `*`를 사용해서 반복할 수 있다. 문자열과 마찬가지로 리스트에서도 되는지 직접 확인해 보자.

리스트 더하기(+)

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> a + b
[1, 2, 3, 4, 5, 6]
```

리스트 사이에서 +는 2개의 리스트를 합치는 기능을 한다. 문자열에서 `"abc" + "def" = "abcdef"`가 되는 것과 같은 이치이다.

리스트 반복하기(`*`)

```
>>> a = [1, 2, 3]
>>> a * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

위에서 볼 수 있듯이 [1, 2, 3] 리스트가 세 번 반복되어 새로운 리스트를 만들어 낸다. 문자열에서 `"abc" * 3 = "abcabcabc"` 가 되는 것과 같은 이치이다.

리스트 길이 구하기

리스트 길이를 구하려면 다음처럼 len 함수를 사용한다.

```
>>> a = [1, 2, 3]
>>> len(a)
3
```

len은 문자열, 리스트 외에 앞으로 배울 튜플과 딕셔너리에도 사용할 수 있는 함수이다. 실습에서 자주 사용하므로 잘 기억해 두자.

점프 투 파이썬

초보자가 범하기 쉬운 리스트 연산 오류

다음 소스 코드를 입력했을 때 결괏값은 어떻게 나올까?

```
>>> a = [1, 2, 3]
>>> a[2] + "hi"
```

a[2]의 값인 3과 문자열 hi가 더해져서 3hi가 출력될 것이라고 생각할 수 있다. 하지만 다음 결과를 보면 오류가 발생했다. 오류의 원인은 무엇일까?

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

a[2]에 저장된 값은 3이라는 정수인데 "hi"는 문자열이다. 정수와 문자열은 서로 더할 수 없기 때문에 오류가 발생한 것이다.

숫자와 문자열을 더해서 '3hi'를 만들고 싶다면 다음처럼 숫자 3을 문자 '3'으로 바꾸어야 한다.

```
>>> str(a[2]) + "hi"
'3hi'
```

str은 정수나 실수를 문자열로 바꾸어 주는 파이썬의 내장 함수이다.

리스트의 수정과 삭제

리스트는 값을 수정하거나 삭제할 수 있다.

리스트의 값 수정하기

```
>>> a = [1, 2, 3]
>>> a[2] = 4
>>> a
[1, 2, 4]
```

a[2]의 요솟값 3이 4로 바뀌었다.

del로 리스트 요소 삭제하기

```
>>> a = [1, 2, 3]
>>> del a[1]
>>> a
[1, 3]
```

`del a[x]`는 x번째 요솟값을 삭제한다. 위에서는 a 리스트에서 a[1]을 삭제했다. del은 파이썬이 기본으로 제공하는 명령어로 객체를 삭제할 때 사용하며 다음과 같이 사용한다.

```
del 객체
```

> *객체란 파이썬에서 사용되는 모든 자료형을 말한다.*

다음처럼 슬라이싱 기법을 사용하여 리스트의 요소 여러 개를 한꺼번에 삭제할 수도 있다.

```
>>> a = [1, 2, 3, 4, 5]
>>> del a[2:]
>>> a
[1, 2]
```

a[2:]에 해당하는 리스트의 요소들이 삭제되었다.

리스트의 요소를 삭제하는 방법에는 2가지가 더 있다. 바로 리스트의 remove와 pop 메서드를 사용하는 것인데, 이는 리스트 관련 함수에서 설명한다.

> *remove, pop 메서드는 리스트에 딸려있는 전용 기능을 뜻한다.*

리스트 관련 함수

문자열과 마찬가지로 리스트 변수 이름 뒤에 '.'를 붙여 여러 가지 리스트 관련 함수를 사용할 수 있다. 유용하게 사용하는 리스트 관련 함수 몇 가지만 알아보자.

리스트에 요소 추가하기 - append

append의 사전적 의미는 '덧붙이다, 첨부하다'이다. 이 뜻을 안다면 다음 예가 바로 이해될 것이다. append(x)는 리스트의 맨 마지막에 x를 추가하는 함수이다.

```
>>> a = [1, 2, 3]
>>> a.append(4)
>>> a
[1, 2, 3, 4]
```

리스트 안에는 어떤 자료형도 추가할 수 있다. 다음 예는 리스트에 다시 리스트를 추가한 결과이다.

```
>>> a.append([5, 6])
>>> a
[1, 2, 3, 4, [5, 6]]
```

리스트 정렬 - sort

sort 함수는 리스트의 요소를 순서대로 정렬해 준다.

```
>>> a = [1, 4, 3, 2]
>>> a.sort()
>>> a
[1, 2, 3, 4]
```

문자 역시 알파벳 순서로 정렬할 수 있다.

```
>>> a = ['a', 'c', 'b']
>>> a.sort()
>>> a
['a', 'b', 'c']
```

리스트 뒤집기 - reverse

reverse 함수는 리스트를 역순으로 뒤집어 준다. 이때 리스트 요소들을 순서대로 정렬한 다음 다시 역순으로 정렬하는 것이 아니라 현재의 리스트를 그대로 거꾸로 뒤집는다.

```
>>> a = ['a', 'c', 'b']
>>> a.reverse()
>>> a
['b', 'c', 'a']
```

인덱스 반환 - index

index(x) 함수는 리스트에 x 값이 있으면 x의 인덱스 값(위칫값)을 반환한다.

```
>>> a = [1, 2, 3]
>>> a.index(3)
2
>>> a.index(1)
0
```

위 예에서 리스트 a에 있는 숫자 3의 위치는 a[2]이므로 2, 숫자 1의 위치는 a[0]이므로 0을 반환한다.

다음 예에서 값 0은 a 리스트에 존재하지 않기 때문에 오류가 발생한다.

```
>>> a.index(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: 0 is not in list
```

리스트에 요소 삽입 - insert

insert(a, b)는 리스트의 a번째 위치에 b를 삽입하는 함수이다. 파이썬은 숫자를 0부터 센다는 것을 반드시 기억하자.

```
>>> a = [1, 2, 3]
>>> a.insert(0, 4)
>>> a
[4, 1, 2, 3]
```

위 예는 0번째 자리, 즉 첫 번째 요소인 a[0] 위치에 값 4를 삽입하라는 뜻이다.

```
>>> a.insert(3, 5)
>>> a
[4, 1, 2, 5, 3]
```

위 예는 리스트 a의 a[3], 즉 네 번째 요소 위치에 값 5를 삽입하라는 뜻이다.

리스트 요소 제거 - remove

remove(x)는 리스트에서 첫 번째로 나오는 x를 삭제하는 함수이다.

```
>>> a = [1, 2, 3, 1, 2, 3]
>>> a.remove(3)
>>> a
[1, 2, 1, 2, 3]
```

a가 3이라는 값을 2개 가지고 있을 경우, 첫 번째 3만 제거되는 것을 알 수 있다.

```
>>> a.remove(3)
>>> a
[1, 2, 1, 2]
```

remove(3)을 한 번 더 실행하면 다시 3이 삭제된다.

리스트 요소 꺼내기 - pop

pop()은 리스트의 맨 마지막 요소를 반환하고 그 요소는 삭제한다.

```
>>> a = [1, 2, 3]
>>> a.pop()
3
>>> a
[1, 2]
```

a 리스트를 다시 보면 [1, 2, 3]에서 3을 꺼내고 최종적으로 [1, 2]만 남는 것을 확인할 수 있다.

pop(x)는 리스트의 x번째 요소를 반환하고 그 요소는 삭제한다.

```
>>> a = [1, 2, 3]
>>> a.pop(1)
2
>>> a
[1, 3]
```

a.pop(1)은 a[1]의 값을 꺼내어 반환한다. 다시 a를 출력해 보면 꺼낸 값이 삭제된 것을 확인할 수 있다.

리스트에 포함된 요소 x의 개수 세기 - count

count(x)는 리스트 안에 x가 몇 개 있는지 조사하여 그 개수를 반환하는 함수이다.

```
>>> a = [1, 2, 3, 1]
>>> a.count(1)
2
```

1이라는 값이 리스트 a에 2개 들어 있으므로 2를 반환한다.

리스트 확장 - extend

extend(x)에서 x에는 리스트만 올 수 있으며 원래의 a 리스트에 x 리스트를 더하게 된다.

```
>>> a = [1, 2, 3]
>>> a.extend([4, 5])
>>> a
[1, 2, 3, 4, 5]
>>> b = [6, 7]
>>> a.extend(b)
>>> a
[1, 2, 3, 4, 5, 6, 7]
```

a.extend([4, 5])는 `a += [4, 5]`와 동일하다.

> *`a += [4, 5]`는 `a = a + [4, 5]`와 동일한 표현식이다.*

02-4 튜플 자료형

튜플(tuple)은 몇 가지 점을 제외하면 리스트와 거의 비슷하다. 리스트와 다른 점은 다음과 같다.

- 리스트는 [], 튜플은 ()으로 둘러싼다.
- 리스트는 요솟값의 생성, 삭제, 수정이 가능하지만, 튜플은 요솟값을 바꿀 수 없다.

- 튜플은 어떻게 만들까?
- 튜플의 요솟값을 지우거나 변경하려고 하면 어떻게 될까?
    - 1. 튜플 요솟값을 삭제하려 할 때
    - 2. 튜플 요솟값을 변경하려 할 때
- 튜플 다루기
    - 인덱싱하기
    - 슬라이싱하기
    - 튜플 더하기
    - 튜플 곱하기
    - 튜플 길이 구하기

튜플은 어떻게 만들까?

튜플은 다음과 같이 여러 가지 모습으로 생성할 수 있다.

```
t1 = ()
t2 = (1,)
t3 = (1, 2, 3)
t4 = 1, 2, 3
t5 = ('a', 'b', ('ab', 'cd'))
```

모습은 리스트와 거의 비슷하지만, 튜플에서는 리스트와 다른 2가지 차이점을 찾아볼 수 있다. `t2 = (1,)` 처럼 단지 1개의 요소만을 가질 때는 요소 뒤에 쉼표(,)를 반드시 붙여야 한다는 것과 `t4 = 1, 2, 3` 처럼 소괄호(())를 생략해도 된다는 점이다.

튜플과 리스트는 비슷해 보이지만, 구별해서 사용하는 것이 좋다. 가장 큰 차이는 요솟값을 변경할 수 있는지 여부이다. 리스트의 요솟값은 변경 가능하고, 튜플의 요솟값은 변경 불가능하다.

프로그램 실행 중 요솟값이 변하지 않기를 바란다면 튜플을 사용하자. 반대로 값을 수시로 변경해야 한다면 리스트를 사용하자. 실제 프로그램에서는 값이 자주 변경되는 경우가 많아 튜플보다 리스트를 더 자주 사용한다.

튜플의 요솟값을 지우거나 변경하려고 하면 어떻게 될까?

튜플의 요솟값은 한 번 정하면 지우거나 변경할 수 없다. 다음 두 예를 살펴보자.

1. 튜플 요솟값을 삭제하려 할 때

```
>>> t1 = (1, 2, 'a', 'b')
>>> del t1[0]
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: 'tuple' object doesn't support item deletion
```

튜플의 요소를 리스트처럼 del 명령어로 지우려고 한 예이다. 튜플은 요솟값을 지울 수 없다는 오류 메시지를 확인할 수 있다.

2. 튜플 요솟값을 변경하려 할 때

```
>>> t1 = (1, 2, 'a', 'b')
>>> t1[0] = 'c'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

튜플의 요솟값을 변경하려고 해도 오류가 발생한다.

튜플 다루기

튜플은 요솟값을 변경할 수 없다는 점만 제외하면 리스트와 동일하다. 다음 예제는 서로 연관되어 있으므로 차례대로 수행해 보자.

인덱싱하기

```
>>> t1 = (1, 2, 'a', 'b')
>>> t1[0]
1
>>> t1[3]
'b'
```

문자열, 리스트와 마찬가지로 t1[0], t1[3]처럼 인덱싱이 가능하다.

슬라이싱하기

```
>>> t1 = (1, 2, 'a', 'b')
>>> t1[1:]
(2, 'a', 'b')
```

t1[1]부터 튜플의 마지막 요소까지 슬라이싱하는 예이다.

튜플 더하기

```
>>> t1 = (1, 2, 'a', 'b')
>>> t2 = (3, 4)
>>> t3 = t1 + t2
>>> t3
(1, 2, 'a', 'b', 3, 4)
```

튜플을 더하는 예이다. 이때 t1, t2 튜플의 요솟값이 바뀌는 것은 아니다. t1, t2 튜플을 더하여 새로운 튜플 t3를 생성한 것이다.

튜플 곱하기

```
>>> t2 = (3, 4)
>>> t3 = t2 * 3
>>> t3
(3, 4, 3, 4, 3, 4)
```

튜플의 곱하기(반복) 예이다.

튜플 길이 구하기

```
>>> t1 = (1, 2, 'a', 'b')
>>> len(t1)
4
```

튜플의 길이를 구하는 예이다.

> *튜플은 요솟값을 변경할 수 없기 때문에 sort, insert, remove, pop과 같은 메서드가 없다.*

02-5 딕셔너리 자료형

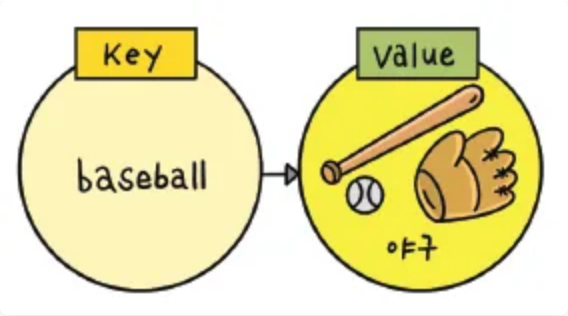
사람은 누구든지 "이름" = "홍길동", "생일" = "몇 월 며칠" 등과 같은 방식으로 그 사람이 가진 정보를 나타낼 수 있다. 파이썬은 영리하게도 이러한 대응 관계를 나타낼 수 있는 딕셔너리(dictionary) 자료형을 가지고 있다. 이번에는 이 자료형에 대해 알아보자.

> *요즘 사용하는 대부분의 언어도 이러한 대응 관계를 나타내는 자료형을 가지고 있는데 이를 딕셔너리라고 하고, '연관 배열(associative array)'또는 '해시(hash)'라고도 한다.*

- 딕셔너리란?
- 딕셔너리는 어떻게 만들까?
- 딕셔너리 쌍 추가, 삭제하기
  - 딕셔너리 쌍 추가하기
  - 딕셔너리 요소 삭제하기
- 딕셔너리를 사용하는 방법
  - 딕셔너리에서 Key를 사용해 Value 얻기
  - 딕셔너리 만들 때 주의할 사항
- 딕셔너리 관련 함수
  - Key 리스트 만들기 - keys
  - Value 리스트 만들기 - values
  - Key, Value 쌍 얻기 - items
  - Key: Value 쌍 모두 지우기 - clear
  - Key로 Value 얻기 - get
  - 해당 Key가 딕셔너리 안에 있는지 조사하기 - in
  - Key로 Value 꺼내기 - pop

딕셔너리란?

딕셔너리는 단어 그대로 '사전'이라는 뜻이다. 즉 "people"이라는 단어에 "사람", "baseball"이라는 단어에 "야구"라는 뜻이 부합되듯이 딕셔너리는 Key와 Value를 한 쌍으로 가지는 자료형이다. 예컨대 Key가 "baseball"이라면 Value는 "야구"가 될 것이다.



딕셔너리는 리스트나 튜플처럼 순차적으로(sequential) 해당 요솟값을 구하지 않고 Key를 통해 Value를 얻는다. 이것이 바로 딕셔너리의 가장 큰 특징이다. baseball이라는 단어의 뜻을 찾기 위해 사전의 내용을 순차적으로 모두 검색하는 것이 아니라 baseball이라는 단어가 있는 곳만 펼쳐 보는 것이다.

> ***파이썬 3.7 이후 딕셔너리 순서 보장***
> *파이썬 3.7 이전에는 딕셔너리에 데이터를 저장할 때 입력한 순서가 보장되지 않았다. 하지만 파이썬 3.7부터는 딕셔너리에 아이템을 삽입한 순서가 유지된다. 즉, 딕셔너리를 순회할 때 아이템이 추가된 순서대로 나온다.*

딕셔너리는 어떻게 만들까?

다음은 딕셔너리의 기본 모습이다.

```
{Key1: Value1, Key2: Value2, Key3: Value3, ... }
```

Key와 Value의 쌍 여러 개가 {}로 둘러싸여 있다. 각각의 요소는 Key: Value 형태로 이루어져 있고 쉼표(,)로 구분되어 있다.

다음 딕셔너리의 예를 살펴보자.

```
>>> dic = {'name': 'pey', 'phone': '010-9999-1234', 'birth': '1118'}
```

위에서 Key는 각각 'name', 'phone', 'birth'이고, 각각의 Key에 해당하는 Value는 'pey', '010-9999-1234', '1118'이다.

딕셔너리 dic의 정보

key	value
name	pey
phone	010-9999-1234
birth	1118

다음은 Key로 정숫값 1, Value로 문자열 'hi'를 사용한 예이다.

```
>>> a = {1: 'hi'}
```

또한 다음 예처럼 Value에 리스트도 넣을 수 있다.

```
>>> a = {'a': [1, 2, 3]}
```

딕셔너리 쌍 추가, 삭제하기

딕셔너리 쌍을 추가하거나 삭제하는 방법을 살펴보자.

딕셔너리 쌍 추가하기

```
>>> a = {1: 'a'}
>>> a[2] = 'b'
>>> a
{1: 'a', 2: 'b'}
```

{1: 'a'} 딕셔너리에 a[2] = 'b'와 같이 입력하면 딕셔너리 a에 Key와 Value가 각각 2와 'b'인 {2: 'b'} 딕셔너리 쌍이 추가된다.

```
>>> a['name'] = 'pey'
>>> a
{1: 'a', 2: 'b', 'name': 'pey'}
```

딕셔너리 a에 {'name': 'pey'} 쌍이 추가되었다.

```
>>> a[3] = [1, 2, 3]
>>> a
{1: 'a', 2: 'b', 'name': 'pey', 3: [1, 2, 3]}
```

Key는 3, Value는 [1, 2, 3]을 가지는 한 쌍이 또 추가되었다.

딕셔너리 요소 삭제하기

```
>>> del a[1]
>>> a
{2: 'b', 'name': 'pey', 3: [1, 2, 3]}
```

위 예제는 딕셔너리 요소를 지우는 방법을 보여 준다. del은 파이썬의 예약어로, `del a[key]`처럼 사용하면 지정한 Key에 해당하는 {Key: Value} 쌍이 삭제된다.

딕셔너리를 사용하는 방법

'딕셔너리는 주로 어떤 것을 표현하는 데 사용할까?'라는 의문이 들 것이다. 예를 들어 4명의 사람이 있다고 가정하고 각자의 특기를 표현할 수 있는 좋은 방법에 대해서 생각해 보자. 리스트나 문자열로는 표현하기가 매우 까다로울 것이다. 하지만 파이썬의 딕셔너리를 사용하면 이 상황을 표현하기가 정말 쉽다.

다음 예를 살펴보자.

```
{"김연아": "피겨스케이팅", "류현진": "야구", "손흥민": "축구", "귀도": "파이썬"}
```

사람 이름과 특기를 한 쌍으로 묶은 딕셔너리이다. 정말 간편하지 않은가?

지금까지 딕셔너리를 만드는 방법을 살펴보았다. 이제 딕셔너리를 제대로 활용하기 위해 알아야 할 것들을 하나씩 알아보자.

딕셔너리에서 Key를 사용해 Value 얻기

```
>>> grade = {'pey': 10, 'julliet': 99}
>>> grade['pey']
10
>>> grade['julliet']
99
```

리스트나 튜플, 문자열은 요솟값을 얻고자 할 때 인덱싱이나 슬라이싱 기법 중 하나를 사용했다. 하지만 딕셔너리는 단 1가지 방법뿐이다. 바로 Key를 사용해서 Value를 구하는 방법이다. 위 예에서 'pey'라는 Key의 Value를 얻기 위해 `grade['pey']`를 사용한 것처럼 어떤 Key의 Value를 얻으려면 '딕셔너리_변수_이름[Key]'를 사용해야 한다.

몇 가지 예를 더 살펴보자.

```
>>> a = {1:'a', 2:'b'}
>>> a[1]
'a'
>>> a[2]
'b'
```

먼저 a 변수에 {1: 'a', 2: 'b'} 딕셔너리를 대입하였다. 위 예에서 볼 수 있듯이 a[1]은 'a' 값을 반환한다.

> ***주의!*** *딕셔너리의 a[1]은 리스트의 a[1]과 전혀 다르다. 리스트에서 a[1]은 "두 번째 요소(인덱스 1)"를 의미하지만, 딕셔너리에서 a[1]은 "Key가 1인 Value"를 의미한다.*

딕셔너리는 리스트나 튜플에 있는 인덱싱 방법을 적용할 수 없다. 따라서 a[1]은 딕셔너리 {1: 'a', 2: 'b'}에서 Key가 1인 것의 Value인 'a'를 반환한다. a[2] 역시 마찬가지이다.

이번에는 a라는 변수에 앞의 예에서 사용한 딕셔너리의 Key와 Value를 뒤집어 놓은 딕셔너리를 대입해 보자.

```
>>> a = {'a': 1, 'b': 2}
>>> a['a']
1
>>> a['b']
2
```

역시 a['a'], a['b']처럼 Key를 사용해서 Value를 얻을 수 있다. 정리하면, 딕셔너리 a는 a[Key]로 Key에 해당하는 Value를 얻는다.

다음 예는 이전에 한 번 언급한 딕셔너리인데, Key를 사용해서 Value를 얻는 방법을 잘 보여 준다.

```
>>> dic = {'name':'pey', 'phone':'010-9999-1234', 'birth': '1118'}
>>> dic['name']
'pey'
>>> dic['phone']
'010-9999-1234'
>>> dic['birth']
'1118'
```

딕셔너리 만들 때 주의할 사항

딕셔너리에서 Key는 고유한 값이므로 중복되는 Key 값을 설정하면 하나를 제외한 나머지 것들이 모두 무시된다. 다음 예에서 볼 수 있듯이 동일한 Key가 2개 존재할 경우, 1: 'a' 쌍이 무시된다.

```
>>> a = {1:'a', 1:'b'}
>>> a
{1: 'b'}
```

Key가 중복되면 1개를 제외한 나머지 Key: Value 값이 모두 무시되는 이유는 Key를 통해서 Value를 얻는 딕셔너리의 특징 때문이다. 즉, 딕셔너리에는 동일한 Key가 중복으로 존재할 수 없다.

또 한 가지 주의할 점은 Key에 리스트는 쓸 수 없다는 것이다. 하지만 튜플은 Key로 쓸 수 있다. 딕셔너리의 Key로 쓸 수 있느냐, 없느냐는 Key가 변하는(mutable) 값인지, 변하지 않는(immutable) 값인지에 달려 있다. 리스트는 그 값이 변할 수 있기 때문에 Key로 쓸 수 없다. 다음 예처럼 리스트를 Key로 설정하면 리스트를 키 값으로 사용할 수 없다는 오류가 발생한다.

```
>>> a = {[1,2] : 'hi'}
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

> *오류 메시지의 "unhashable"은 "해시할 수 없다"는 뜻으로, 변경 가능한(mutable) 자료형은 딕셔너리의 Key로 사용할 수 없다는 의미이다.*

단, Value에는 변하는 값이든, 변하지 않는 값이든 아무 값이나 넣을 수 있다.

딕셔너리 관련 함수

딕셔너리를 자유자재로 사용하기 위해 딕셔너리가 자체적으로 가지고 있는 관련 함수를 사용해 보자.

Key 리스트 만들기 - keys

```
>>> a = {'name': 'pey', 'phone': '010-9999-1234', 'birth': '1118'}
>>> a.keys()
dict_keys(['name', 'phone', 'birth'])
```

a.keys()는 딕셔너리 a의 Key만을 모아 dict_keys 객체를 반환한다. dict_keys는 리스트와 비슷하게 사용할 수 있는 객체로, for 문에서 바로 사용하거나 `list()` 함수로 리스트로 변환할 수 있다.

점프 투 파이썬

dict_keys 객체와 리스트 변환

keys, values, items 함수는 리스트가 아닌 dict_keys, dict_values, dict_items 객체를 반환한다. 이 객체들은 for 문 등 반복 구문에서 리스트처럼 사용할 수 있다. 만약 리스트가 필요한 경우에는 `list(a.keys())` 처럼 변환하면 된다.

dict_keys 객체는 다음과 같이 사용할 수 있다. 리스트를 사용하는 것과 별 차이는 없지만, 리스트 고유의 append, insert, pop, remove, sort 함수는 수행할 수 없다.

```
>>> for k in a.keys():
...     print(k)
...
name
phone
birth
```

> - *print(k)를 입력할 때 들여쓰기를 하지 않으면 오류가 발생하므로 주의하자.*
> - *for 문 등 반복 구문에 대해서는 03장에서 자세히 살펴본다.*

dict_keys 객체를 리스트로 변환하려면 다음과 같이 하면 된다.

```
>>> list(a.keys())
['name', 'phone', 'birth']
```

Value 리스트 만들기 - values

```
>>> a.values()
dict_values(['pey', '010-9999-1234', '1118'])
```

Key를 얻는 것과 마찬가지 방법으로 Value만 얻고 싶다면 values 함수를 사용하면 된다. values 함수를 호출하면 dict_values 객체를 반환한다.

Key, Value 쌍 얻기 - items

```
>>> a.items()
dict_items([('name', 'pey'), ('phone', '010-9999-1234'), ('birth', '1118')])
```

items 함수는 Key와 Value의 쌍을 튜플로 묶은 값을 dict_items 객체로 반환한다.

Key: Value 쌍 모두 지우기 - clear

```
>>> a.clear()
>>> a
{}
```

clear 함수는 딕셔너리 안의 모든 요소를 삭제한다.

> *빈 리스트를 [], 빈 튜플을 ()로 표현하는 것과 마찬가지로 빈 딕셔너리도 {}로 표현한다.*

Key로 Value 얻기 - get

```
>>> a = {'name': 'pey', 'phone': '010-9999-1234', 'birth': '1118'}
>>> a.get('name')
'pey'
>>> a.get('phone')
'010-9999-1234'
```

get(x) 함수는 x라는 Key에 대응되는 Value를 반환한다. 앞에서 살펴보았듯이 `a.get('name')`은 `a['name']`을 사용했을 때와 동일한 결괏값을 반환한다.

다만, 다음 예제에서 볼 수 있듯이 `a['nokey']`처럼 딕셔너리에 존재하지 않는 키로 값을 가져오려고 할 경우, `a['nokey']` 방식은 오류를 발생시키고 `a.get('nokey')` 방식은 None을 반환한다는 차이가 있다. Key가 반드시 존재한다고 확신할 수 있을 때는 `a[key]` 방식을, Key가 없을 수도 있을 때는 `a.get(key)` 방식을 사용하면 된다.

> *여기에서 None은 값이 없음을 나타내는 객체이며, 조건문에서는 False처럼 평가된다.*

```
>>> a = {'name':'pey', 'phone':'010-9999-1234', 'birth': '1118'}
>>> a.get('nokey')
None
>>> a['nokey']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'nokey'
```

딕셔너리 안에 찾으려는 Key가 없을 경우, 미리 정해 둔 디폴트 값을 대신 가져오게 하고 싶을 때는 get(x, '디폴트 값')을 사용하면 편리하다.

```
>>> a.get('nokey', '정보없음')
'정보없음'
```

딕셔너리 a에는 'nokey'에 해당하는 Key가 없다. 따라서 디폴트 값인 '정보없음'을 반환한다.

해당 Key가 딕셔너리 안에 있는지 조사하기 - in

```
>>> a = {'name':'pey', 'phone':'010-9999-1234', 'birth': '1118'}
>>> 'name' in a
True
>>> 'email' in a
False
```

'name' 문자열은 a 딕셔너리의 Key 중 하나이다. 따라서 'name' in a를 호출하면 참(True)을 반환한다. 반대로 'email'은 a 딕셔너리 안에 존재하지 않는 Key이므로 거짓(False)을 반환한다.

Key로 Value 꺼내기 - pop

```
>>> a = {'name':'pey', 'phone':'010-9999-1234', 'birth': '1118'}
>>> phone = a.pop('phone')
>>> phone
'010-9999-1234'
>>> a
{'name': 'pey', 'birth': '1118'}
```

pop(x) 함수는 딕셔너리에서 Key가 x인 항목을 삭제한 후 그 Value를 반환한다. get 함수와 달리 pop은 해당 항목을 딕셔너리에서 삭제한다는 차이가 있다. 존재하지 않는 Key에 대해서는 디폴트 값을 지정할 수 있다.

```
>>> email = a.pop('email', '정보없음')
>>> email
'정보없음'
```

02-6 집합 자료형

집합(set)은 수학의 집합 개념과 동일한 자료형이다. 중복을 허용하지 않고 순서가 없는 데이터의 모임으로, 교집합, 합집합, 차집합 등의 집합 연산을 쉽게 처리할 수 있다.

- 집합 자료형은 어떻게 만들까?
- 집합 자료형의 특징
- 교집합, 합집합, 차집합 구하기
    - 교집합 구하기
    - 합집합 구하기
    - 차집합 구하기
- 집합 자료형 관련 함수
    - 값 1개 추가하기 - add
    - 값 여러 개 추가하기 - update
    - 특정 값 제거하기 - remove
    - 특정 값 제거하기 - discard
    - 모든 값 제거하기 - clear

집합 자료형은 어떻게 만들까?

집합 자료형은 다음과 같이 set 함수를 사용해 만들 수 있다.

```
>>> s1 = set([1, 2, 3])
>>> s1
{1, 2, 3}
```

set()의 괄호 안에 리스트를 입력하거나, 다음과 같이 문자열을 입력해서 만들 수도 있다.

```
>>> s2 = set("Hello")
>>> s2
{'e', 'H', 'l', 'o'}
```

또한 중괄호({})를 사용해서 집합 자료형을 직접 만들 수도 있다.

```
>>> s3 = {1, 2, 3}
>>> s3
{1, 2, 3}
>>> s4 = {'a', 'b', 'c'}
>>> s4
{'a', 'c', 'b'}
```

> *비어 있는 집합 자료형은 `s = set()` 로 만들 수 있다. `s = {}` 로 만들면 딕셔너리가 되므로 주의하자.*

집합 자료형의 특징

그런데 위에서 살펴본 `set("Hello")` 의 결과가 좀 이상하지 않은가? 분명 "Hello" 문자열로 집합 자료형을 만들었는데 생성된 자료형에는 l 문자가 하나 빠져 있고 순서도 뒤죽박죽이다. 그 이유는 집합에 다음과 같은 2가지 특징이 있기 때문이다.

- 중복을 허용하지 않는다.
- 순서가 없다(Unordered).

> *집합은 중복을 허용하지 않는 특징 때문에 데이터의 중복을 제거하는 필터로 종종 사용된다.*

리스트나 튜플은 순서가 있기(ordered) 때문에 인덱싱으로 요솟값을 얻을 수 있지만, 집합 자료형은 순서가 없기(unordered) 때문에 인덱싱으로 요솟값을 얻을 수 없다.

만약 집합 자료형에 저장된 값을 인덱싱으로 접근하려면 다음과 같이 리스트나 튜플로 변환한 후에 해야 한다.

```
>>> s1 = set([1, 2, 3])
>>> l1 = list(s1)
>>> l1
[1, 2, 3]
>>> l1[0]
1
>>> t1 = tuple(s1)
>>> t1
(1, 2, 3)
>>> t1[0]
1
```

교집합, 합집합, 차집합 구하기

집합 자료형을 정말 유용하게 사용하는 경우는 교집합, 합집합, 차집합을 구할 때이다.

먼저 다음과 같이 2개의 집합을 만들어 보자. s1에는 1부터 6까지의 값, s2에는 4부터 9까지의 값을 넣었다.

```
>>> s1 = set([1, 2, 3, 4, 5, 6])
>>> s2 = set([4, 5, 6, 7, 8, 9])
```

교집합 구하기

s1과 s2의 교집합을 구해 보자. `&`를 사용하면 교집합을 간단히 구할 수 있다.

```
>>> s1 & s2
{4, 5, 6}
```

intersection 함수를 사용해도 동일한 결과를 얻는다.

```
>>> s1.intersection(s2)
{4, 5, 6}
```

s2.intersection(s1)을 사용해도 결과는 동일하다.

합집합 구하기

이번에는 합집합을 구해 보자. `|`를 사용하면 합집합을 구할 수 있다. 4, 5, 6처럼 중복된 값은 1개씩만 표현된다.

```
>>> s1 | s2
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

union 함수를 사용해도 된다.

```
>>> s1.union(s2)
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

s2.union(s1)을 사용해도 결과는 동일하다.

차집합 구하기

마지막으로 차집합을 구해 보자. `-`(빼기)를 사용하면 차집합을 구할 수 있다.

```
>>> s1 - s2
{1, 2, 3}
>>> s2 - s1
{8, 9, 7}
```

difference 함수를 사용해도 차집합을 구할 수 있다.

```
>>> s1.difference(s2)
{1, 2, 3}
>>> s2.difference(s1)
{8, 9, 7}
```

집합 자료형 관련 함수

값 1개 추가하기 - add

이미 만들어진 집합에 값을 추가할 수 있다. 1개의 값만 추가할 때는 다음과 같이 하면 된다.

```
>>> s1 = set([1, 2, 3])
>>> s1.add(4)
>>> s1
{1, 2, 3, 4}
```

값 여러 개 추가하기 - update

여러 개의 값을 한꺼번에 추가할 때는 다음과 같이 하면 된다.

```
>>> s1 = set([1, 2, 3])
>>> s1.update([4, 5, 6])
>>> s1
{1, 2, 3, 4, 5, 6}
```

특정 값 제거하기 - remove

특정 값을 제거하고 싶을 때는 다음과 같이 하면 된다.

```
>>> s1 = set([1, 2, 3])
>>> s1.remove(2)
>>> s1
{1, 3}
```

특정 값 제거하기 - discard

remove와 비슷하지만, 존재하지 않는 값을 제거하려 해도 오류가 발생하지 않는다.

```
>>> s1 = set([1, 2, 3])
>>> s1.discard(2)
>>> s1
{1, 3}
>>> s1.discard(4)  # 없는 값이어도 오류가 발생하지 않음
>>> s1
{1, 3}
```

모든 값 제거하기 - clear

집합의 모든 값을 제거할 때는 clear 함수를 사용한다.

```
>>> s1 = set([1, 2, 3])
>>> s1.clear()
>>> s1
set()
```

02-7 불 자료형

불(bool) 자료형이란 참(True)과 거짓(False)을 나타내는 자료형이다. 불 자료형은 다음 2가지 값만을 가질 수 있다.

- True: 참을 의미한다.
- False: 거짓을 의미한다.

> *True나 False는 파이썬의 예약어로, true, false와 같이 작성하면 안 되고 첫 문자를 항상 대문자로 작성해야 한다.*

- 불 자료형은 어떻게 사용할까?
- 자료형의 참과 거짓
- 불 연산
- 논리 연산자
    - and 연산자
    - or 연산자
    - not 연산자
    - 실제 활용 예제

불 자료형은 어떻게 사용할까?

다음과 같이 변수 a에는 True, 변수 b에는 False를 지정해 보자.

```
>>> a = True
>>> b = False
```

따옴표로 감싸지 않은 문자열을 변수에 지정해서 오류가 발생할 것 같지만, 잘 실행된다. type 함수로 변수 a와 b를 확인하면 자료형이 bool인 것을 알 수 있다.

```
>>> type(a)
<class 'bool'>
>>> type(b)
<class 'bool'>
```

> *type(x)는 x의 자료형을 확인하는 파이썬의 내장 함수이다.*

불 자료형은 조건문의 반환값으로도 사용된다. 조건문은 if 문에서 자세히 배우겠지만 잠시 살펴보고 넘어가자.

```
>>> 1 == 1
True
```

`1 == 1` 은 '1과 1이 같은가?'를 묻는 조건문이다. 이런 조건문은 결과로 True 또는 False에 해당하는 불 자료형을 반환한다. 1과 1은 같으므로 True를 반환한다.

```
>>> 2 > 1
True
```

2는 1보다 크므로 `2 > 1` 조건문은 참이다. 즉, True를 반환한다.

```
>>> 2 < 1
False
```

2는 1보다 작지 않으므로 `2 < 1` 조건문은 거짓이다. 즉, False를 반환한다.

자료형의 참과 거짓

'자료형에 참과 거짓이 있다?'라는 말이 조금 이상하게 들리겠지만, 참과 거짓은 분명히 있다. 이는 매우 중요한 특징이며 실제로도 자주 쓰인다.

자료형의 참과 거짓을 구분하는 기준은 다음과 같다.

값	참 or 거짓
"python"	참
""	거짓
[1, 2, 3]	참
[]	거짓
(1, 2, 3)	참
()	거짓
{'a': 1}	참
{}	거짓
1	참
0	거짓
None	거짓

문자열, 리스트, 튜플, 딕셔너리 등의 값이 비어 있으면("", [], (), {}) 거짓이 되고 비어 있지 않으면 참이 된다. 숫자에서는 그 값이 0일 때 거짓이 된다. 위 표에 있는 None은 뒷부분에서 배우므로 아직은 신경 쓰지 말자. 그저 None은 값이 없음을 나타내는 객체이며, 참/거짓을 따질 때는 False처럼 동작한다는 것만 알아 두자.

다음 예를 보고 자료형의 참과 거짓이 프로그램에서 어떻게 쓰이는지 간단히 알아보자.

```
>>> a = [1, 2, 3, 4]
>>> while a:
...     print(a.pop())
...
4
3
2
1
```

먼저 `a = [1, 2, 3, 4]` 라는 리스트를 만들었다. while 문은 03장에서 자세히 다루지만, 간단히 알아보면 다음과 같다. 조건문이 참인 동안 조건문 안에 있는 문장을 반복해서 수행한다.

```
while 조건문:
    수행할_문장
```

위 예는 a가 참인 경우 a.pop()을 계속 실행하여 출력하라는 의미이다. a.pop() 함수는 리스트 a의 마지막 요소를 꺼내는 함수이므로 리스트 안에 요소가 있는 한(a가 참인 동안) 마지막 요소를 계속 꺼낸다. 결국 더 이상 꺼낼 것이 없으면 a가 빈 리스트([])가 되어 거짓이 된다. 따라서 while 문에서 조건문이 거짓이 되므로 while 문을 빠져나가게 된다. 이는 파이썬 프로그래밍에서 매우 자주 사용하는 기법 중 하나이다.

위 예가 너무 복잡하다면 다음 예를 보자.

```
>>> if []:
...     print("참")
... else:
...     print("거짓")
...
거짓
```

[]는 앞의 표에서 볼 수 있듯이 비어 있는 리스트이므로 거짓이다. 따라서 "거짓"이라는 문자열이 출력된다. if 문에 대해서 잘 모르는 독자라도 위 문장을 해석하는 데는 무리가 없을 것이다.

> *if 문에 대해서는 03장에서 자세히 다룬다.*

다른 예도 하나만 더 살펴보자.

```
>>> if [1, 2, 3]:
...     print("참")
... else:
...     print("거짓")
...
참
```

이 코드를 해석해 보면 다음과 같다.

```
만약 [1, 2, 3]이 참이면 "참"이라는 문자열을 출력하고, 그렇지 않으면 "거짓"이라는 문자열을 출력하라.
```

[1, 2, 3]은 요솟값이 있는 리스트이므로 참이다. 따라서 "참"을 출력한다.

불 연산

bool 함수를 사용하면 자료형의 참과 거짓을 정확하게 확인할 수 있다.

다음 예제를 따라 해 보자.

```
>>> bool('python')
True
```

'python' 문자열은 비어 있지 않으므로 bool 연산의 결과로 True를 반환한다.

```
>>> bool('')
False
```

'' 문자열은 비어 있으므로 bool 연산의 결과로 False를 반환한다.

앞에서 알아본 몇 가지 예제를 더 수행해 보자.

```
>>> bool([1, 2, 3])
True
>>> bool([])
False
>>> bool(0)
False
>>> bool(3)
True
```

앞에서 알아본 것과 동일한 참과 거짓에 대한 결과를 반환하는 것을 확인할 수 있다.

논리 연산자

불 자료형에서는 and, or, not과 같은 논리 연산자를 사용할 수 있다.

and 연산자

and 연산자는 양쪽 조건이 모두 참일 때만 True를 반환한다.

```
>>> True and True
True
>>> True and False
False
>>> False and True
False
>>> False and False
False
```

or 연산자

or 연산자는 양쪽 조건 중 하나라도 참이면 True를 반환한다.

```
>>> True or True
True
>>> True or False
True
>>> False or True
True
>>> False or False
False
```

not 연산자

not 연산자는 참과 거짓을 반대로 바꾼다.

```
>>> not True
False
>>> not False
True
>>> not 1
False
>>> not 0
True
```

실제 활용 예제

```
>>> x = 5
>>> y = 10
>>> x > 0 and y > 0
True
>>> x > 10 or y > 5
True
>>> not (x > y)
True
```

지금까지 파이썬의 가장 기본이 되는 자료형인 숫자, 문자열, 리스트, 튜플, 딕셔너리, 집합, 불에 대해서 알아보았다. 여기까지 잘 따라온 독자라면 파이썬에 대해서 대략 50% 정도 습득했다고 보아도 된다. 그만큼 자료형은 중요하고 프로그램의 근간이 되기 때문에 확실하게 해 놓지 않으면 좋은 프로그램을 만들 수 없다. 이 책의 예제만 따라 하지 말고 직접 여러 가지 예들을 테스트해 보면서 02-1~02-7에 나오는 자료형에 익숙해지기 바란다.

02-8 자료형의 값을 저장하는 공간, 변수

변수의 기본 개념은 쉽게 이해할 수 있지만, 이 챕터의 후반부에서 다루는 메모리와 객체 참조에 관한 내용은 프로그래밍 초보자에게 다소 어려울 수 있다. 당장 이해되지 않는 부분이 있다면 그냥 읽고 지나가도 된다. 파이썬을 더 사용하다 보면 자연스럽게 이해하게 될 것이다.

- 변수는 어떻게 만들까?
- 변수 명명 규칙
  - 변수명 규칙
  - 파이썬 예약어
  - 변수명 권장 사항
- 변수란?
- 리스트를 복사할 때
  - 1. [:] 이용하기
  - 2. copy 모듈 이용하기
- 변수를 만드는 여러 가지 방법

변수는 어떻게 만들까?

우리는 앞에서 이미 변수를 사용해 왔다. 다음 예와 같은 a, b, c를 '변수'라고 한다.

```
>>> a = 1
>>> b = "python"
>>> c = [1, 2, 3]
```

변수를 만들 때는 위 예처럼 =(assignment) 기호를 사용한다.

```
변수_이름 = 변수에_저장할_값
```

C나 JAVA 같은 언어에서는 변수를 만들 때 자료형을 직접 지정해야 한다. 하지만 파이썬은 변수에 저장된 값을 스스로 판단하여 자료형을 지정하므로 더 편리하다.

변수 명명 규칙

변수를 만들 때는 다음 규칙을 지켜야 한다.

변수명 규칙

1. 영문자, 숫자, 언더스코어(_)만 사용할 수 있다.
2. 숫자로 시작할 수 없다.
3. 예약어는 사용할 수 없다.
4. 대소문자를 구분한다.

```
# 올바른 변수명 예시
name = "홍길동"
age = 25
user_name = "gildong"
userName = "gildong"  # 카멜 케이스
_private = "비공개"
count1 = 10

# 잘못된 변수명 예시
# 1name = "홍길동"    # 숫자로 시작 (오류)
# user-name = "홍길동"  # 하이픈 사용 (오류)
# if = 10             # 예약어 사용 (오류)
```

파이썬 예약어

파이썬에서 특별한 의미로 사용되는 예약어들은 변수명으로 사용할 수 없다.

```
False, None, True, and, as, assert, break, class, continue, def,
del, elif, else, except, finally, for, from, global, if, import,
in, is, lambda, nonlocal, not, or, pass, raise, return, try,
while, with, yield
```

변수명 권장 사항

- 의미가 명확한 이름을 사용한다.
- snake_case(단어 사이에 언더스코어)를 권장한다.
- 너무 짧거나 긴 이름은 피한다.

```
# 좋은 예
student_name = "김철수"
total_score = 95
user_age = 20

# 피해야 할 예
a = "김철수"  # 의미 불명확
studentNameFromKorea = "김철수"  # 너무 긴 이름
```

변수란?

파이썬에서 변수는 객체를 가리키는 것이라고 말할 수 있다. 객체란 우리가 지금까지 보아 온 자료형의 데이터(값)와 같은 것을 의미한다. 예를 들어 숫자 1, 문자열 'python', 리스트 [1, 2, 3] 모두 객체이다(객체에 대해서는 05-1절에서 자세히 공부한다).

변수와 객체의 관계를 쉽게 이해하려면 변수를 이름표라고 생각하면 된다. 물건에 이름표를 붙이듯이 데이터(객체)에 변수라는 이름표를 붙여 두면, 나중에 그 이름표를 통해 데이터를 찾을 수 있다.

```
a = [1, 2, 3]
```

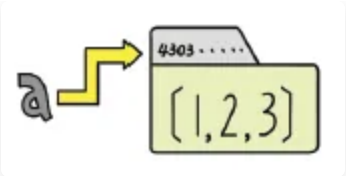
위 코드처럼 `a = [1, 2, 3]` 이라고 하면 [1, 2, 3] 값을 가지는 리스트 객체가 자동으로 메모리에 생성되고, 변수 a는 [1, 2, 3] 리스트가 저장된 메모리의 주소를 가리키게 된다.

> *메모리란 컴퓨터가 프로그램에서 사용하는 데이터를 기억하는 공간이다.*

a 변수가 가리키는 메모리의 주소는 다음과 같이 확인할 수 있다.

```
>>> a = [1, 2, 3]
>>> id(a)
4303029896
```

id는 변수가 가리키는 객체의 주소 값을 반환하는 파이썬 내장 함수이다. 위 예에서 변수 a가 가리키는 [1, 2, 3] 리스트의 주소 값은 43030298896이다.



리스트를 복사할 때

리스트 자료형에서 가장 혼동하기 쉬운 '복사'에 대해 알아보자. 이 개념을 모르면 "분명히 a만 바꿨는데 b도 바뀌어 있다"는 예상치 못한 버그를 만나게 되므로 꼭 이해하고 넘어가자.

```
>>> a = [1, 2, 3]
>>> b = a
```

b 변수에 a 변수를 대입하면 어떻게 될까? b와 a는 같을까, 다를까? 결론부터 말하면 b는 a와 완전히 동일하다. 다만 [1, 2, 3] 이라는 리스트 객체를 참조하는 변수가 a 하나에서 b가 추가되어 2개로 늘어났다는 차이만 있을 뿐이다.

id 함수를 사용하면 이 사실을 확인할 수 있다.

```
>>> id(a)
4303029896
>>> id(b)
4303029896
```

id(a)의 값이 id(b)의 값과 동일하다. 즉, a가 가리키는 대상과 b가 가리키는 대상이 같다. 동일한 객체를 가리키는지 판단하는 is 명령어를 사용해도 참을 반환한다.

```
>>> a is b  # a와 b가 가리키는 객체가 같을까?
True
```

다음 예를 계속 수행해 보자.

```
>>> a[1] = 4
>>> a
[1, 4, 3]
>>> b
[1, 4, 3]
```

a 리스트의 두 번째 요소를 4로 바꾸었더니 a만 바뀌는 것이 아니라 b도 똑같이 바뀌었다. a, b 모두 동일한 리스트를 가리키고 있기 때문이다.

그렇다면 b 변수를 생성할 때 a 변수의 값을 가져오면서 a와는 다른 주소를 가리키도록 만들 수는 없을까? 다음 2가지 방법이 있다.

1. [:] 이용하기

첫 번째 방법은 리스트 전체를 가리키는 [:]을 사용해서 복사하는 것이다.

```
>>> a = [1, 2, 3]
>>> b = a[:]
>>> a[1] = 4
>>> a
[1, 4, 3]
>>> b
[1, 2, 3]
```

a 리스트 값을 바꾸더라도 b 리스트에는 영향이 없다.

2. copy 모듈 이용하기

두 번째 방법은 copy 모듈을 사용하는 것이다. 다음 예에서 `from copy import copy` 라는 처음 보는 문장이 나오는데, 이것은 뒤에서 다룰 파이썬 모듈 부분에서 자세히 설명한다. 여기서는 copy 함수를 쓰기 위한 것이라고만 알아 두자.

```
>>> from copy import copy
>>> a = [1, 2, 3]
>>> b = copy(a)
```

위 예에서 b = copy(a)는 b = a[:]과 동일하다.

두 변수의 값은 같지만, 서로 다른 객체를 가리키는지 확인해 보자.

```
>>> b is a
False
```

`b is a` 가 False를 반환하므로 b와 a가 가리키는 객체는 서로 다르다.

점프 투 파이썬

copy 함수 사용하기

리스트 자료형의 자체 함수인 copy 함수를 사용해도 copy 모듈을 사용하는 것과 동일한 결과를 얻을 수 있다.

```
>>> a = [1, 2, 3]
>>> b = a.copy()
>>> b is a
False
```

> *리스트를 복사하는 방법으로 `b = a[:]`, `b = copy(a)`, `b = a.copy()` 세 가지를 배웠다. 셋 다 동일한 결과를 얻을 수 있지만, 별도의 모듈을 가져올 필요 없이 간편하게 사용할 수 있는 `b = a[:]` 또는 `b = a.copy()` 를 주로 사용한다.*

변수를 만드는 여러 가지 방법

튜플로 a, b에 값을 대입할 수 있다.

```
>>> a, b = ('python', 'life')
```

이 방법은 다음 예문과 완전히 동일하다.

```
>>> (a, b) = 'python', 'life'
```

튜플 부분에서도 언급했지만, 튜플은 괄호를 생략해도 된다.

리스트로 변수를 만들 수도 있다.

```
>>> [a, b] = ['python', 'life']
```

여러 개의 변수에 같은 값을 대입할 수도 있다.

```
>>> a = b = 'python'
```

파이썬에서는 이 방법을 사용하여 두 변수의 값을 간단하게 바꿀 수 있다.

```
>>> a = 3
>>> b = 5
>>> a, b = b, a
>>> a
5
>>> b
3
```

처음에 a에 값 3, b에는 값 5가 대입되어 있었지만 `a, b = b, a` 문장을 수행한 후에는 그 값이 서로 바뀌었다. 다른 프로그래밍 언어에서는 두 변수의 값을 바꾸려면 임시 변수가 필요하지만, 파이썬에서는 이처럼 한 줄로 간단히 처리할 수 있다.

02장 되새김 문제

죄송합니다. 되새김 문제는 위키독스 웹 사이트에서 비공개 처리되어 있습니다.
되새김 문제에 대한 상세한 내용은 유료 E-book 또는 종이책에서 확인할 수 있습니다.

- 책 구입 안내